

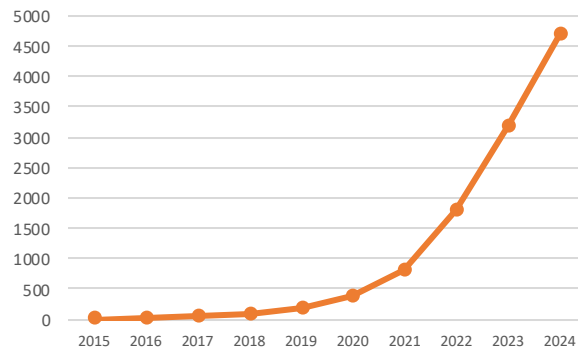
Physics-Informed Machine Learning

Marc Sebban

Hubert Curien Lab – UMR CNRS 5516

MALICE Inria team

Jean Monnet University



papers with title containing
"Physics-informed ML" (source Googles Scholar)



Quentin BERTRAND
Inria Junior
Researcher



Eduardo BRANDAO
Associate Professor



Naima CHALAIS TRAORE
Research Team
Assistant



Farah CHERFAOUI
Associate Professor



Rémi EMONET
Associate Professor
(HDR)
Junior IUF Member



Rémi EYRAUD
Associate Professor
(HDR)



Benjamin GIRAULT
Inria Junior
Researcher



Amaury HABRARD
Professor
Senior IUF Member



Jordan PATRACONE
Associate Professor



Marc SEBBAN
Professor
Head of the MALICE
team



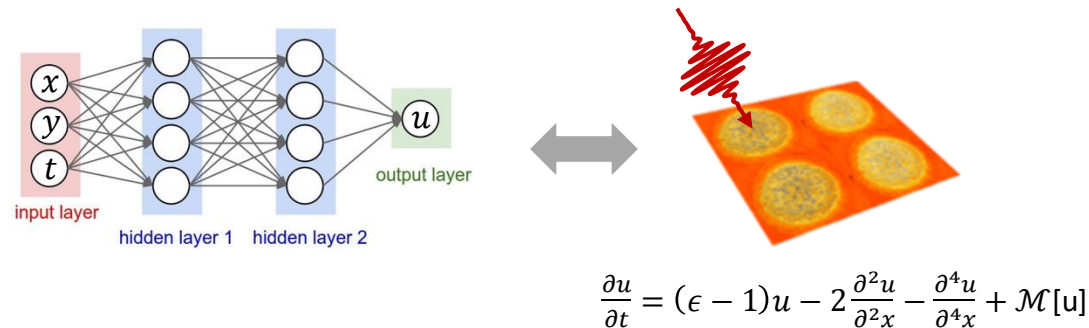
Outline

1- Introduction

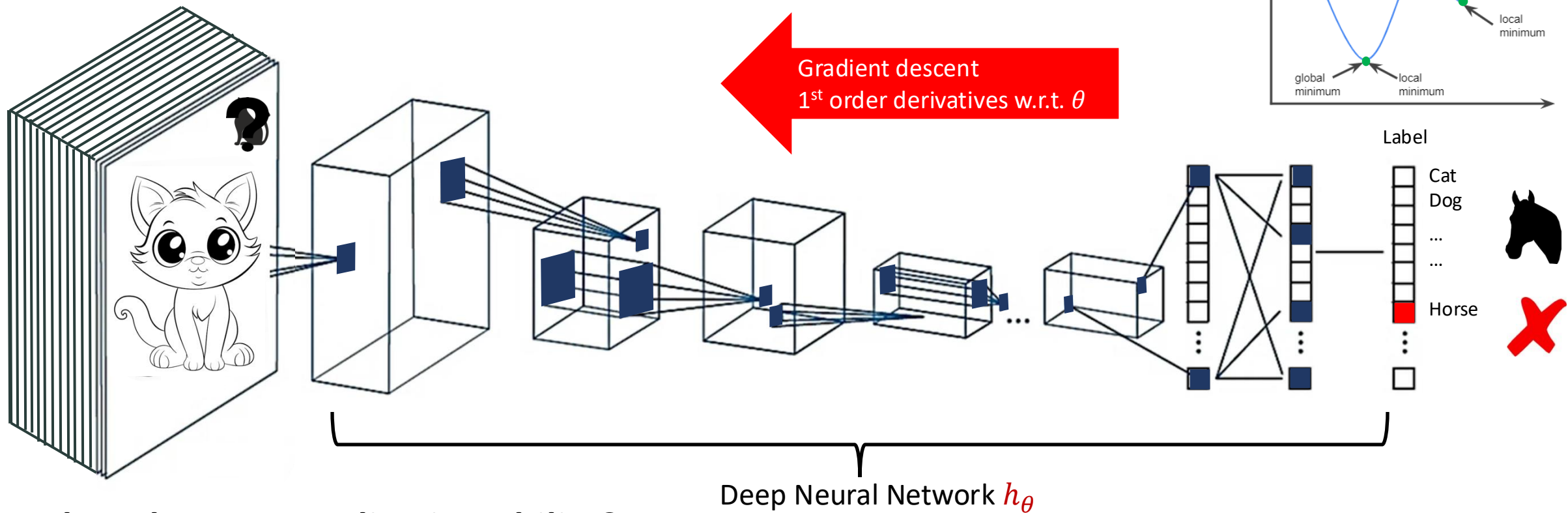
2- Overview of some Physics-informed ML algorithms (PINNs, FNO, PINO, ...)

3- PDE Discovery (SINDy and its variants) **(Topic of the MLDM Project!)**

4- Physics-informed ML and Laser-Matter interaction (what we do at Hubert Curien Lab)



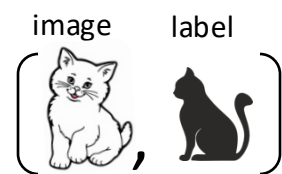
Machine Learning (Toy example 1)



What about generalization ability?



Learning requires a large amount of data



Regularized Risk Minimization

To prevent overfitting

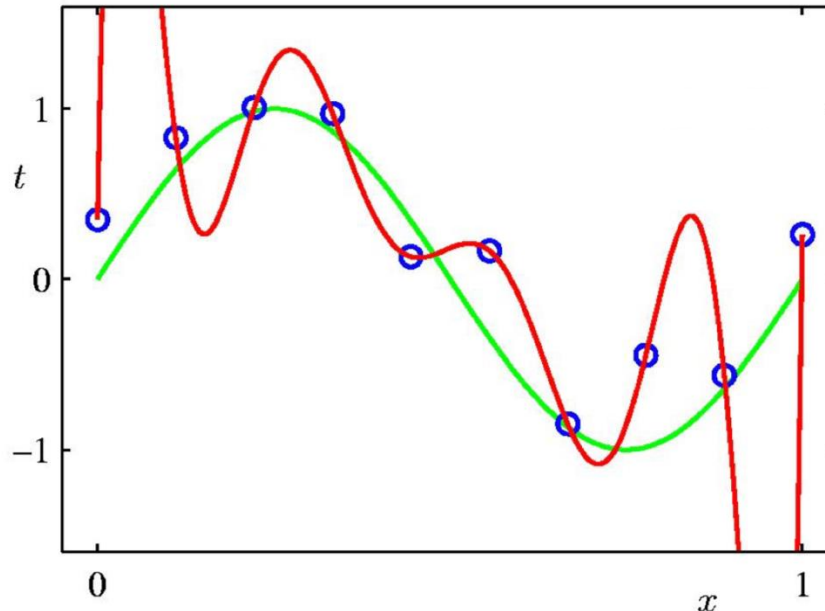
$$\min_{h_{\theta} \in \mathcal{H}} \hat{\mathcal{R}}^{\ell}(h_{\theta}) + \lambda \text{Reg}(\theta)$$

Penalizes a large norm

≈ Controls the magnitude of the coefficients

Regularization also allows us:

- to correct ill-posed problems
- to converge faster
- to inject side-information, prior knowledge on the problem

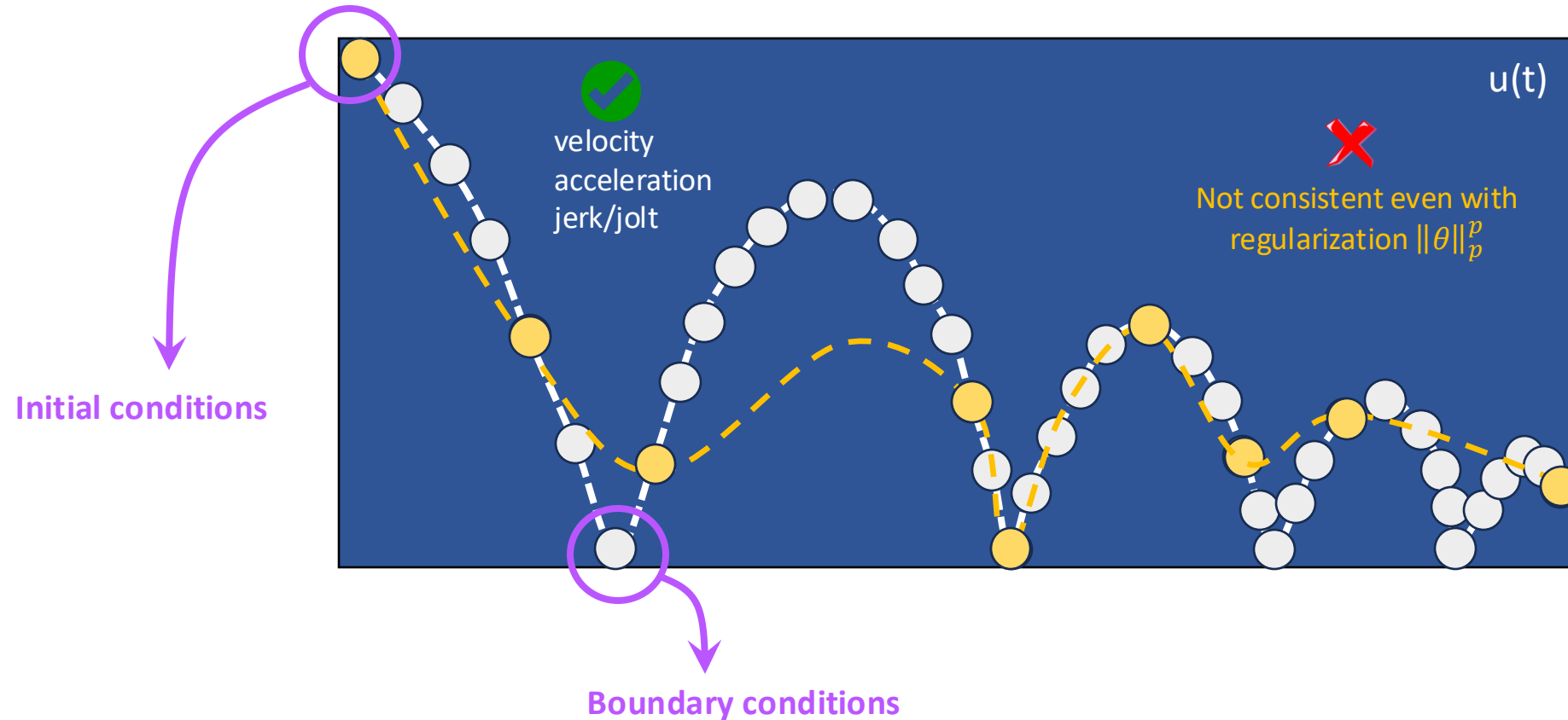


Is there prior knowledge available?

Yes, a lot in physics!

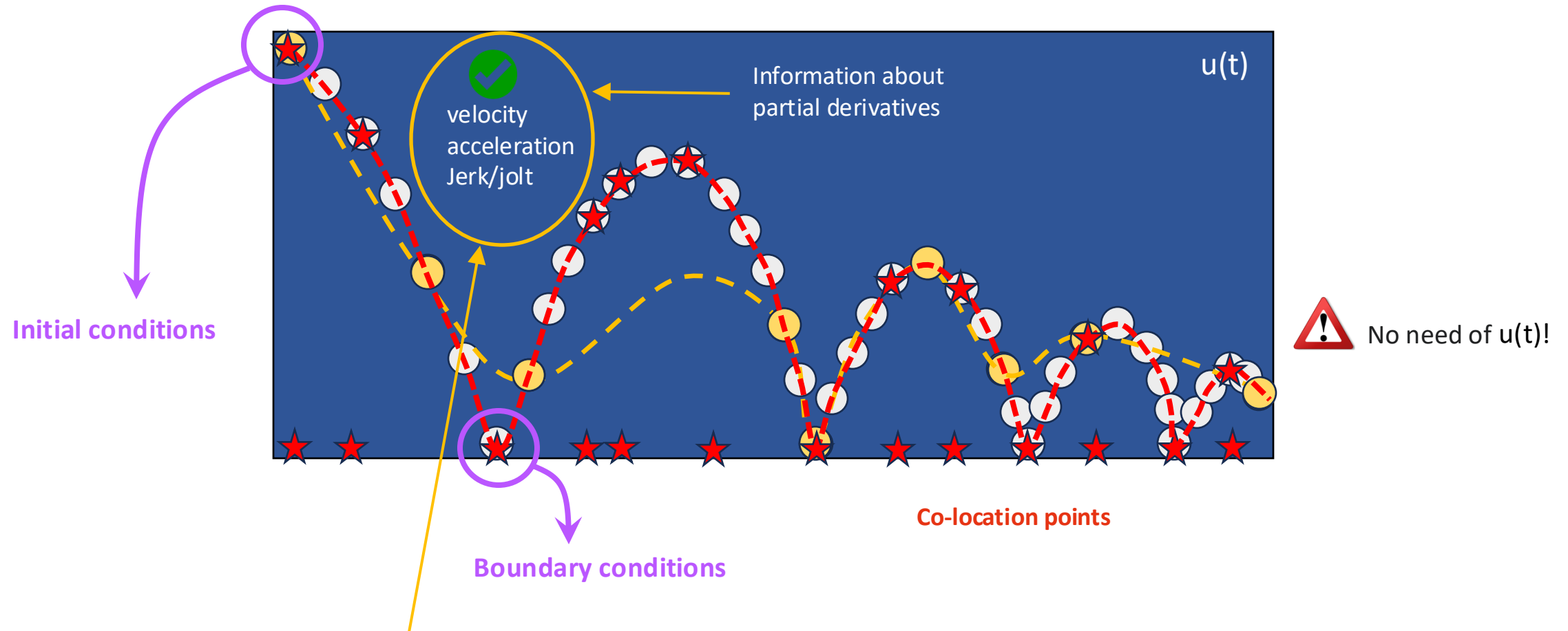
Machine Learning

(Toy example 2: *the motion of a bouncing ball*)



Machine Learning

(Toy example 2: *the motion of a bouncing ball*)



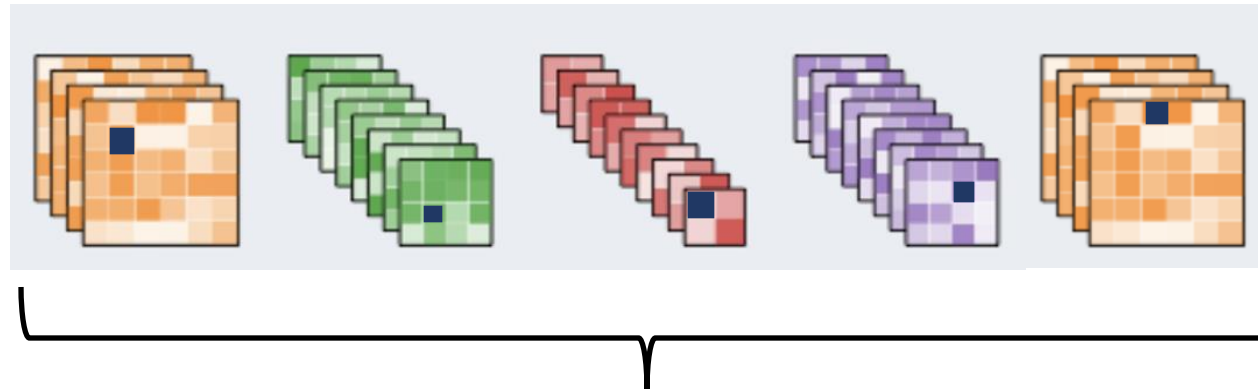
How is this information about partial derivatives represented?

Machine Learning

(Toy example 3: fluid mechanics)



time T



More challenging: learning requires computing n^{th} order derivatives of $\hat{u}_\theta$

This prediction does not satisfy the physical laws!

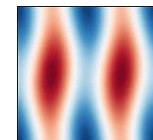


$T + \delta t$?

“Physics-informed Machine Learning”



The learning process is based on both **data**
...and **physical laws** (ODEs, PDEs)



Laplacian

$$\begin{aligned} \dot{u} + (u \cdot \nabla)u &= -\nabla p + \nu \Delta u \\ \nabla \cdot u &= 0 \end{aligned}$$

Physical knowledge $\approx$ ODE or PDE

Example of ODE

(Ordinal Differential Equation)

Newton's second law of motion

$$m \frac{\partial^2 u(t)}{\partial t^2} = F(u(t))$$

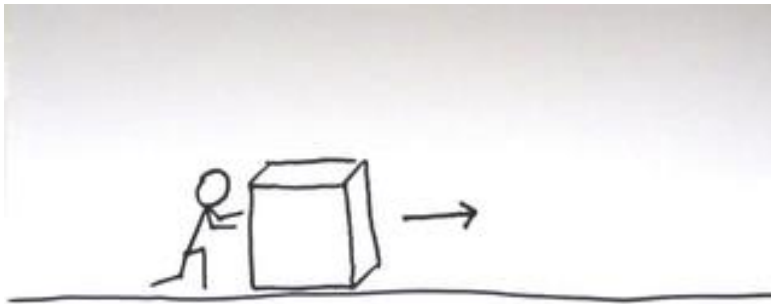
Mass

Acceleration

(second derivative)

Force

Here, u depends on only one *independent* variable



This law tells us that the acceleration is proportional to the force applied, where the constant of proportionality is the mass

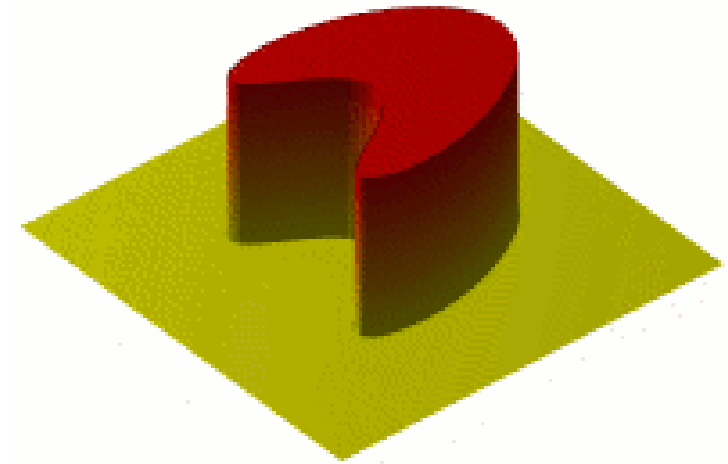
Example of PDE

(Partial Differential Equation)

Heat Equation

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right)$$

Here, u depends on t , x , y and z



As time passes the heat (the temperature at each point is described by the height and redness) diffuses into the cold region (in green)

Allen–Cahn Equation (PDE)

Well-known PDE from the area of reaction–diffusion systems

$$u_t - 0.0001u_{xx} + 5u^3 - 5u = 0, \quad x \in [-1, 1], \quad t \in [0, 1],$$

Where $u_t = \frac{\partial u(t,x)}{\partial t}$

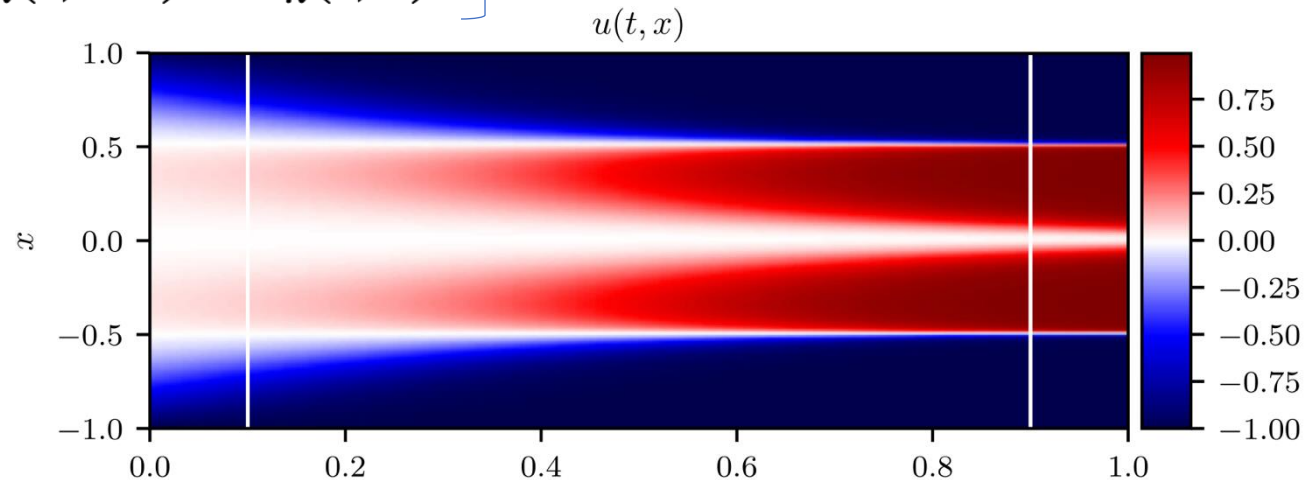
Initial conditions $u(0, x) = x^2 \cos(\pi x),$

Boundary conditions $\left. \begin{aligned} u(t, -1) &= u(t, 1), \\ u_x(t, -1) &= u_x(t, 1). \end{aligned} \right\}$

Dirichlet BC: specify the value of u

Neumann BC: specify the values of the derivative

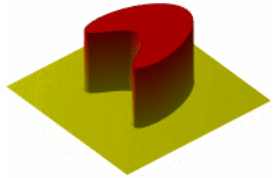
and $u_{xx} = \frac{\partial^2 u(t,x)}{\partial x^2}$



What situations can benefit from physical knowledge?

Different Scenarios of Physics-Informed ML

No need of data



High

Physical Models

Amount of Background Knowledge

Low

Low

Amount of Data

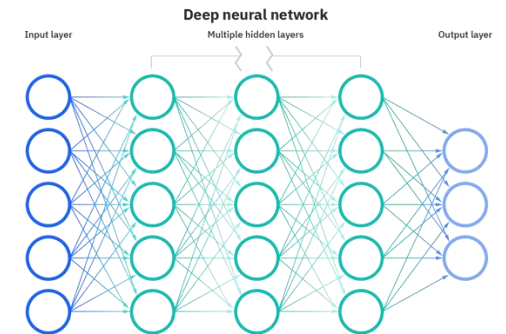
Physics-Informed Machine Learning

Machine Learning

« “All models are wrong, but some are useful.”
(George Box)

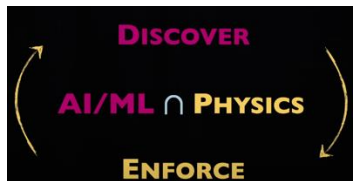
The theory tells us that

$$\mathcal{R}(h) \leq \hat{\mathcal{R}}(h) + \sqrt{\frac{\text{“Complexity Measure”}}{m}}$$



No need of physical theory

High



Physics-informed Machine Learning

A « win-win » combination



Through the lens of physics: ML has emerged as a promising way to

- Learn neural **PDE solvers** (PINN) that are **much faster** than numerical solvers
- **Augment/Discover** knowledge about the underlying physics
- **Generalize** to different (initial/boundary) conditions/resolutions (FNO)

$$\begin{aligned}\dot{u} + (u \cdot \nabla)u &= -\nabla p + \nu \Delta u \\ \nabla \cdot u &= 0\end{aligned}$$



Through the lens of machine learning: Physics helps to...

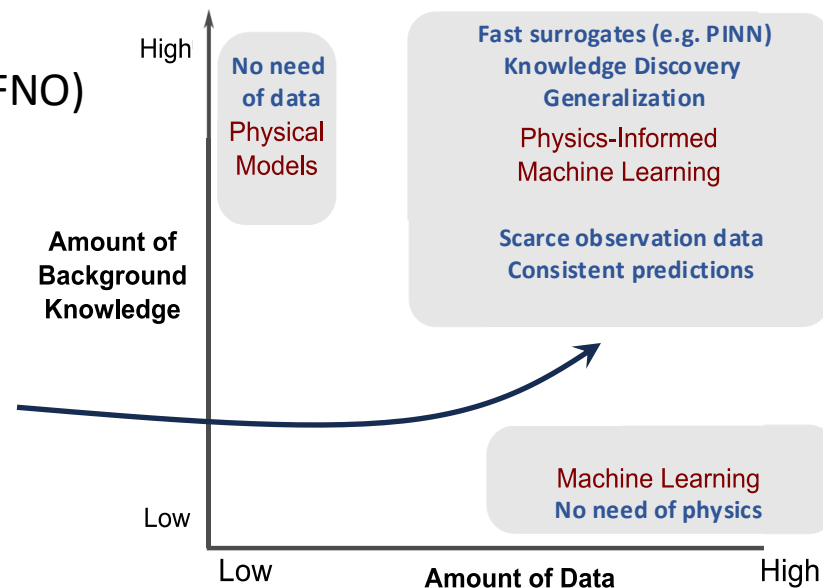
- **Correct ill-posed** optimization problems
- Converge **faster**
- Provide **consistent predictions** when **data is scarce**

$$\min_{\theta} \mathcal{L}(\theta, S) + \lambda \text{Reg}(\theta)$$

standard $\text{Reg}(\theta) = \|\theta\|_p^p$
is not enough!

Seminal PIML papers:

- Neural ODE [NeurIPS 2018]
- PINN [JCP 2019]
- FNO [ICLR 2021]



Scenarios in PIML

Outline

1- Introduction

2- Overview of some Physics-informed ML algorithms (PINNs, FNO, PINO, ...)

3- PDE Discovery (SINDy and its variants)

4- Physics-informed ML and Laser-Matter interaction (what we do at Hubert Curien Lab)

Learning of a PDE solver

PINNs: Physics-Informed Neural Networks

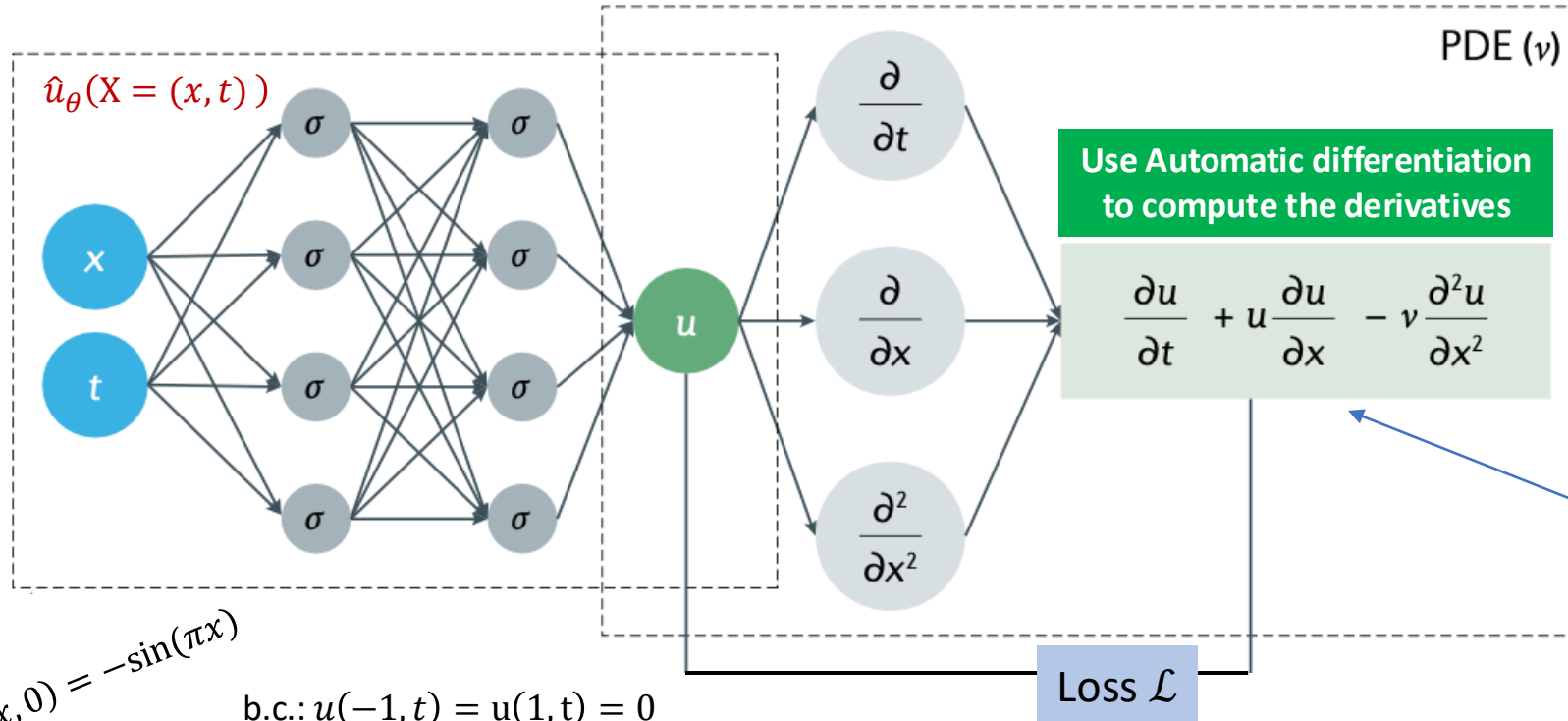
(Raissi et al. 2019)

PINNs: Physics-informed Neural Nets

(Raissi et al. 2019)

Goal: learn a fast (hopefully accurate) solver of one instance of a PDE

✓ In its original version: No need of labeled data/simulations (just sampled points)



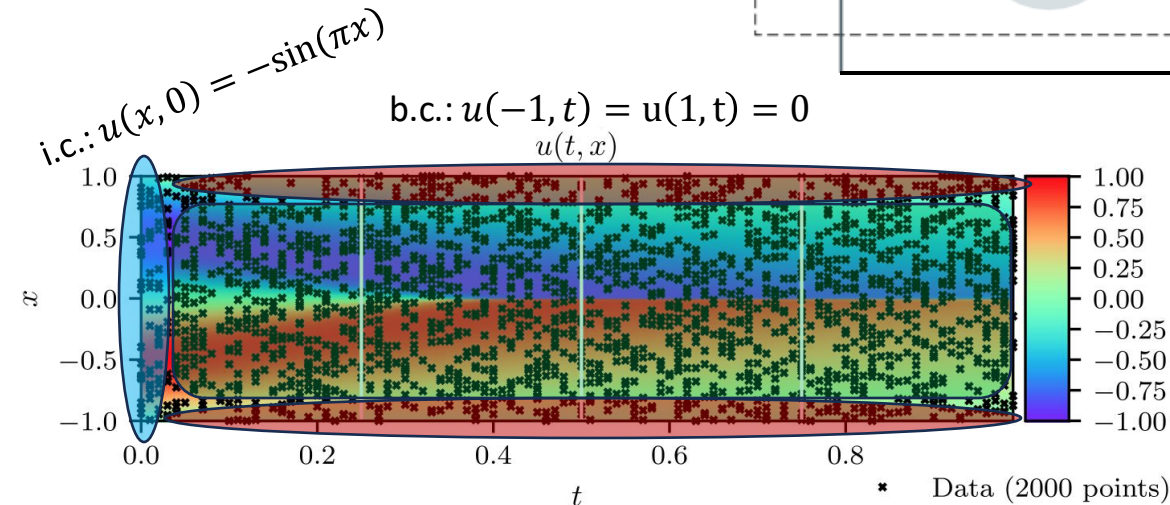
Example with Burgers' Equation, where:

- ν is the viscosity
 - u is the speed
- (comes from fluid mechanics)

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

i.c.: $u(x, 0) = -\sin(\pi x)$

b.c.: $u(-1, t) = u(1, t) = 0$



Points sampled at
initial boundary
conditions

Co-location points

$$\mathcal{L}(\hat{u}_\theta) = \frac{\lambda_s}{n_s} \sum_{j=1}^{n_s} \left\| \hat{u}_\theta(X_j) - u(X_j) \right\|_2^2 + \frac{1}{n_{PDE}} \sum_{i=1}^{n_{PDE}} \left\| \frac{\partial \hat{u}}{\partial t} + \hat{u} \frac{\partial \hat{u}}{\partial x} - \nu \frac{\partial^2 \hat{u}}{\partial x^2} \right\|_2^2$$

residuals

Hybrid PINN (observation data+physical knowledge) (Boyer et al. 2024)

« To make the best of both worlds »

$$\mathcal{L}(\hat{u}_\theta) = \frac{\lambda_s}{n_s} \sum_{j=1}^{n_s} \|\hat{u}_\theta(X_j) - u(X_j)\|_2^2$$

n_s sampled **initial points** $X = (x, 0)$
and **boundary points** $X = (1, t), X = (-1, t)$

$$+ \frac{1}{n_{PDE}} \sum_{k=1}^{n_{PDE}} \left\| \frac{\partial \hat{u}_\theta(X_k)}{\partial t} + \hat{u}_\theta(X_k) \frac{\partial \hat{u}_\theta(X_k)}{\partial x} - \nu \frac{\partial^2 \hat{u}_\theta(X_k)}{\partial x^2} \right\|_2^2$$

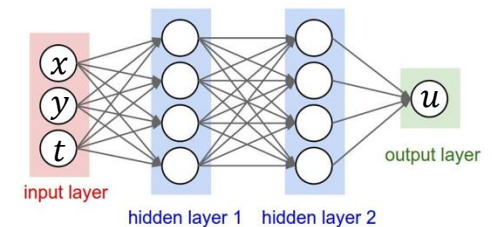
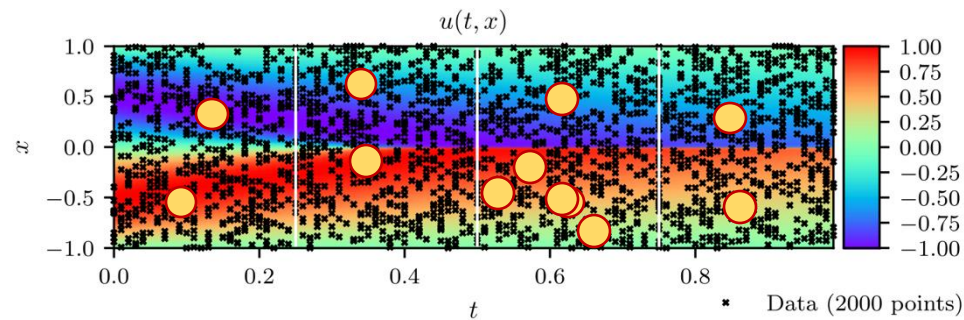
n_{PDE} sampled **co-location points** $X = (x, t)$

$$+ \frac{\lambda_d}{n_l} \sum_{i=1}^{n_l} \|\hat{u}_\theta(X_i) - u(X_i)\|_2^2$$

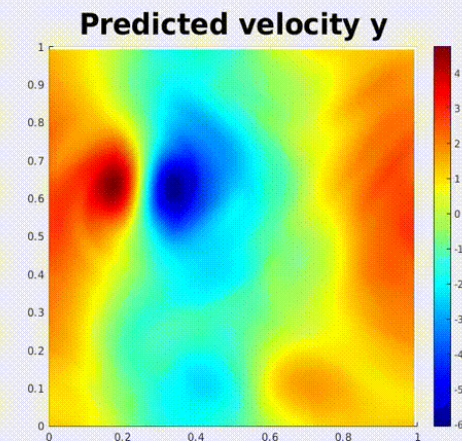
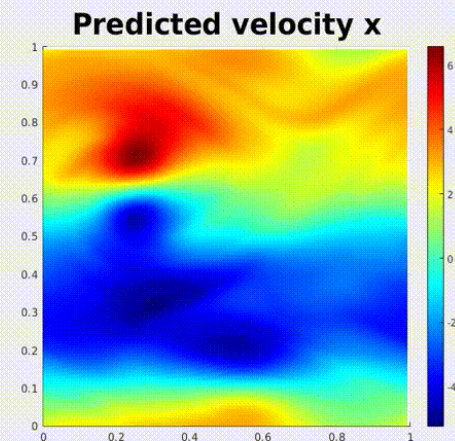
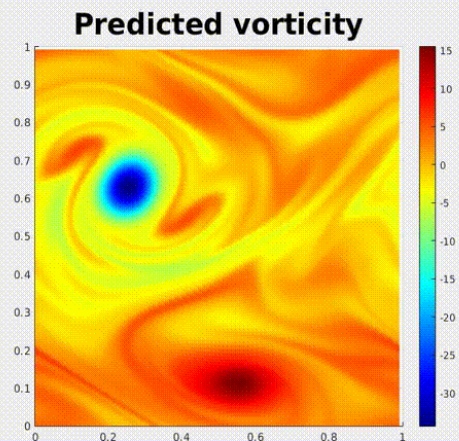
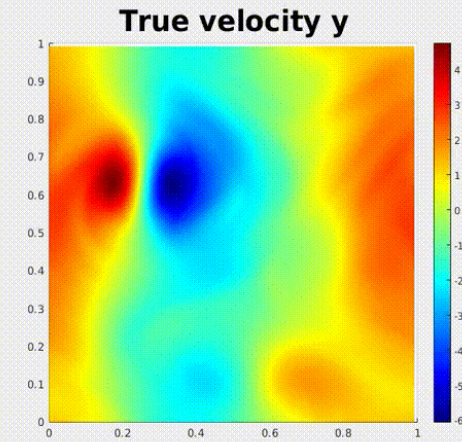
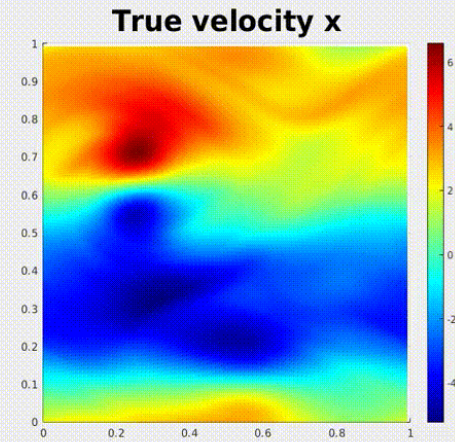
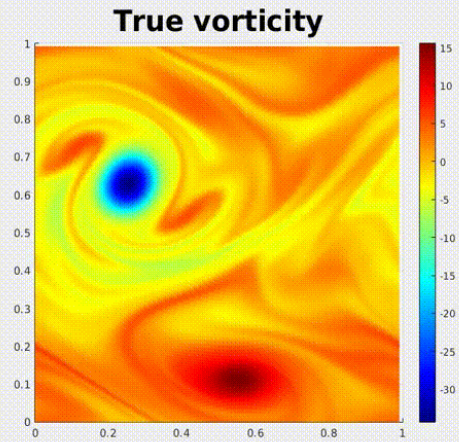
n_l labeled **observation data** $((x, t), u(x, t))$

Karniadakis et al. 2021

“Despite the empirical success of physics-informed learning models, little is known about the theoretical foundation (...) A new theory is required to rigorously analyse the capabilities and limitations of physics-informed learning”.



Example where it works very well!



Pathological behavior of PINNs:

Inconsistent $\hat{u}_\theta$ with exploding derivatives (Boyer et al. 2023)

Consider the heat propagation differential operator on $\Omega =]-1; +1[\times]0; T[$:

$$\frac{\partial u(x)}{\partial t} = \frac{\partial^2 u(x)}{\partial x^2} \quad \text{and} \quad u(-1, t) = u(1, t) = 0 \quad \forall t \in [0; T]$$

Inconsistent PINN that predicts zero on Ω (and thus satisfies the PDE!), while satisfying the initial condition. This network $\hat{u}_\theta$ has a $\|\theta\|_2 \rightarrow \infty$

No guarantee that
small residual $\approx$ small error on $\hat{u}_\theta$

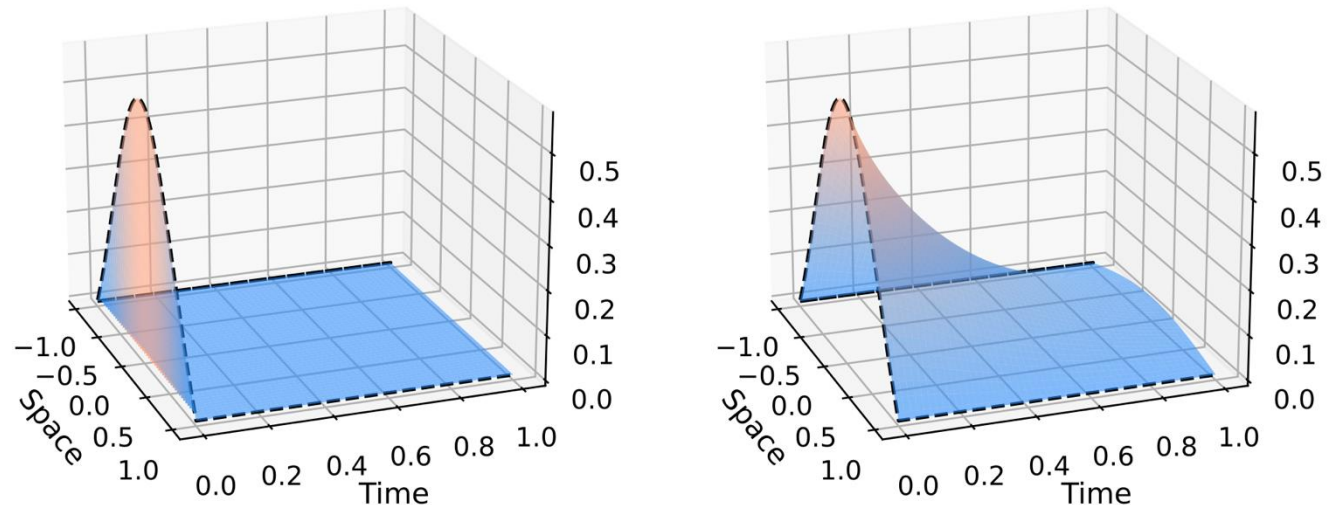
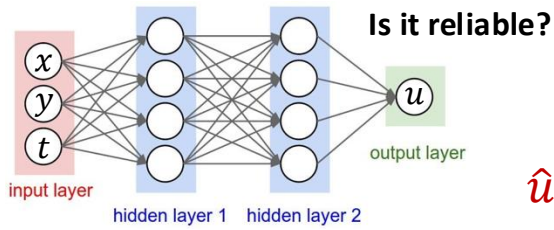


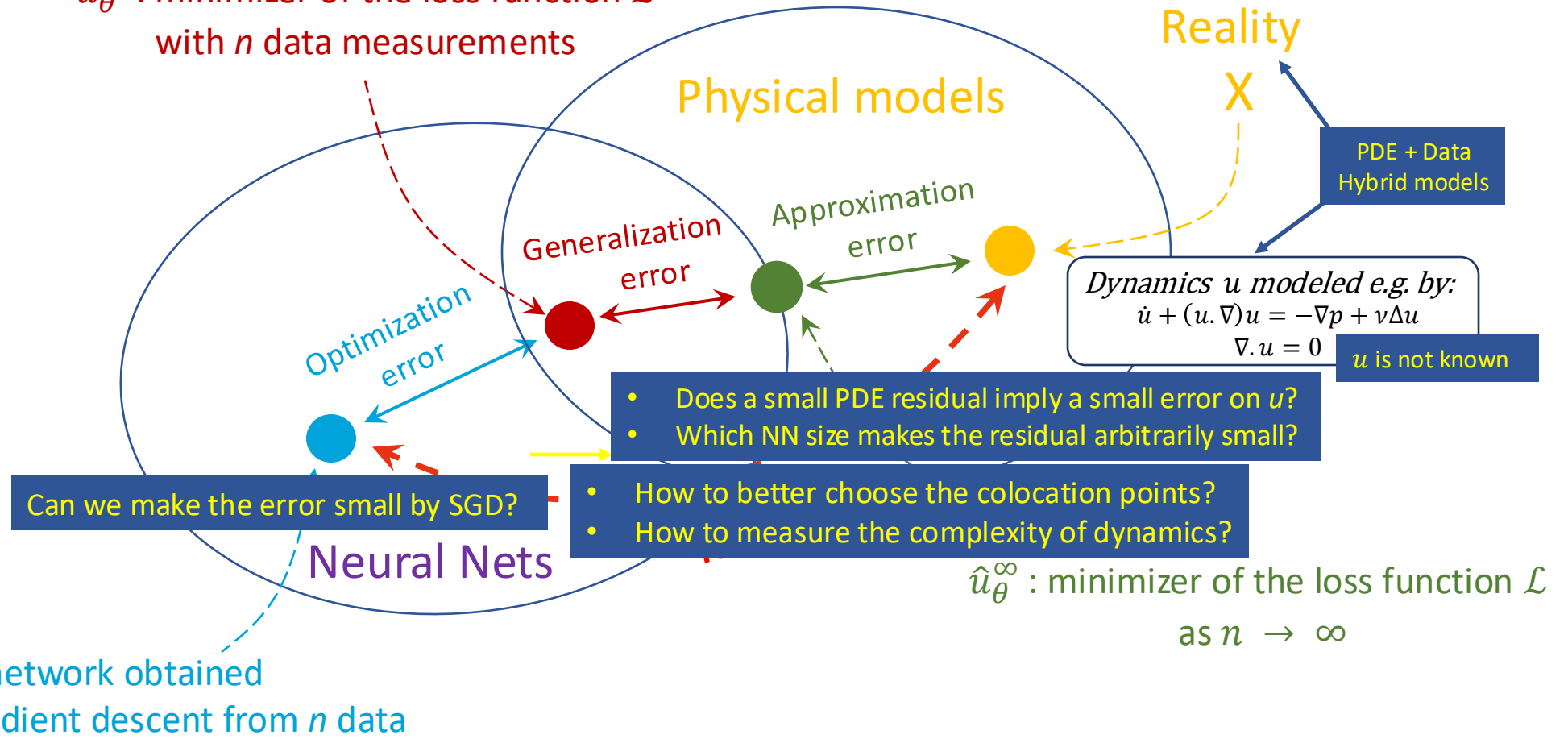
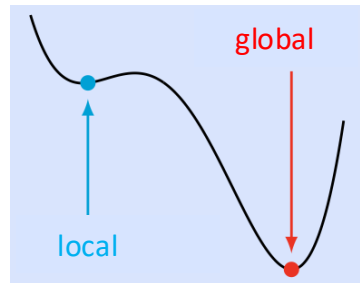
FIG 2. Inconsistent PINN (left) compared to the solution u^* of the PDE (right) for the heat propagation case.



Theoretical Questions in PIML

Error Analysis of PINN $\hat{u}_\theta$

$\hat{u}_\theta^n$: minimizer of the loss function $\mathcal{L}$
with n data measurements



Back to the differentiation in PINNs...

Automatic Differentiation

The goal of *autodiff* is to obtain the numerical value of the derivative.

Underlying idea : differentiable functions are composed of underlying primitive operations whose derivatives are known.

The chain rule allows us to compose these together!

Forward Mode

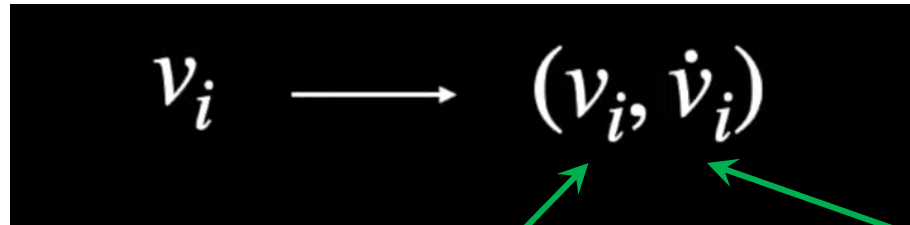
Reverse Mode

Differentiation

Automatic Differentiation

Forward Mode

Forward *autodiff* involves augmenting each intermediate variable during the evaluation of a function with its derivative.


$$v_i \longrightarrow (v_i, \dot{v}_i)$$

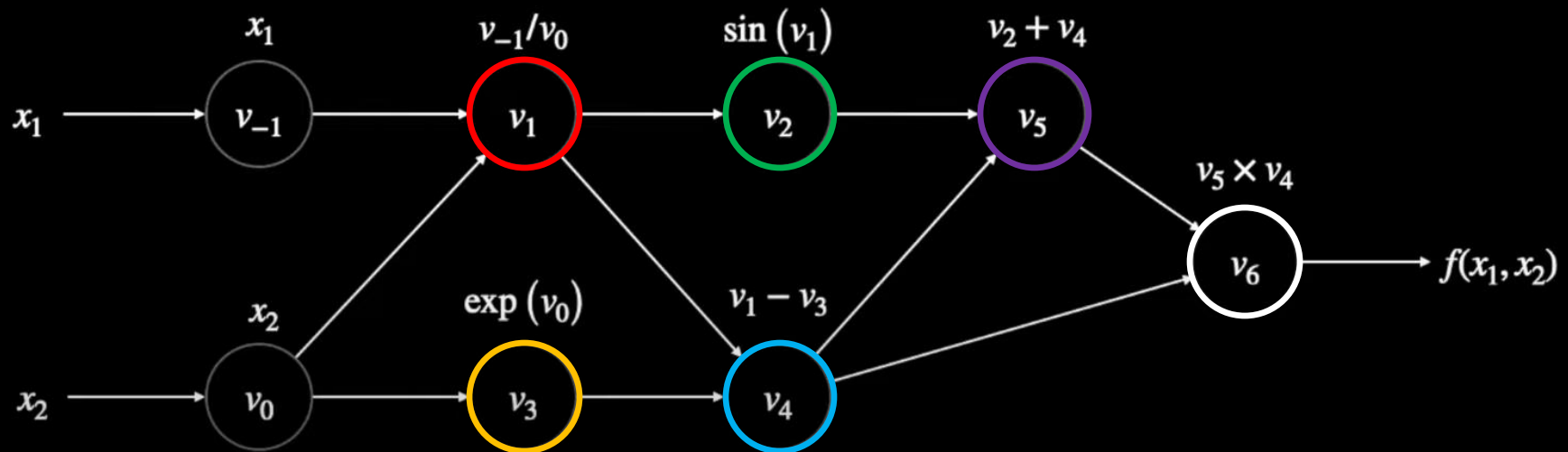
Primal paired with its derivative (known as **tangent**)

Differentiation

Automatic Differentiation

Forward Mode : example

$$f(x_1, x_2) = \left[\sin\left(\frac{x_1}{x_2}\right) + \frac{x_1}{x_2} - e^{x_2} \right] \times \left[\frac{x_1}{x_2} - e^{x_2} \right]$$

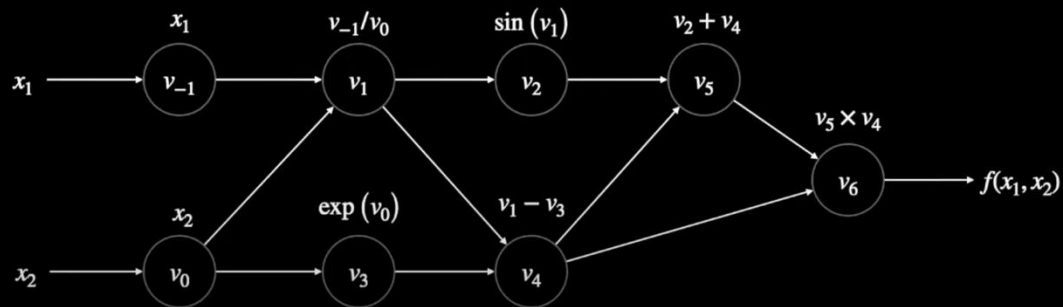


Differentiation

Automatic Differentiation

Forward Mode

$$\frac{\partial f}{\partial x_1} \text{ at } (1.5, 0.5)?$$



Primals

$v_{-1} = x_1$	$= 1.500$
$v_0 = x_2$	$= 0.500$
$v_1 = v_{-1}/v_0$	$= 3.000$
$v_2 = \sin(v_1)$	$= 0.141$
$v_3 = \exp(v_0)$	$= 1.649$
$v_4 = v_1 - v_3$	$= 1.351$
$v_5 = v_2 + v_4$	$= 1.492$
$v_6 = v_5 \times v_4$	$= 2.017$
$f(x_1, x_2) = v_6$	$= 2.017$

Tangents $\frac{\partial v_i}{\partial x_1}$

$\dot{v}_{-1}$	$= 1.000$
$\dot{v}_0$	$= 0.000$
$\dot{v}_1 = (v_0 \dot{v}_{-1} - v_{-1} \dot{v}_0)/v_0^2$	$= 2.000$
$\dot{v}_2 = \cos(v_1) \times \dot{v}_1$	$= -1.980$
$\dot{v}_3 = v_3 \times \dot{v}_0$	$= 0.000$
$\dot{v}_4 = \dot{v}_1 - \dot{v}_3$	$= 2.000$
$\dot{v}_5 = \dot{v}_2 + \dot{v}_4$	$= 0.020$
$\dot{v}_6 = \dot{v}_5 v_4 - \dot{v}_4 v_5$	$= 3.012$
$\frac{\partial f}{\partial x_1} = \dot{v}_6$	$= 3.012$

Differentiation

Automatic Differentiation

Forward Mode and Jacobian Evaluation

Each pass of the forward mode produces one column of the Jacobian (derivatives w.r.t. 1 variable)

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m$$

forward mode is ideal when $n \ll m$

$$J_f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Not suitable for ML problems
Where $n \gg m$

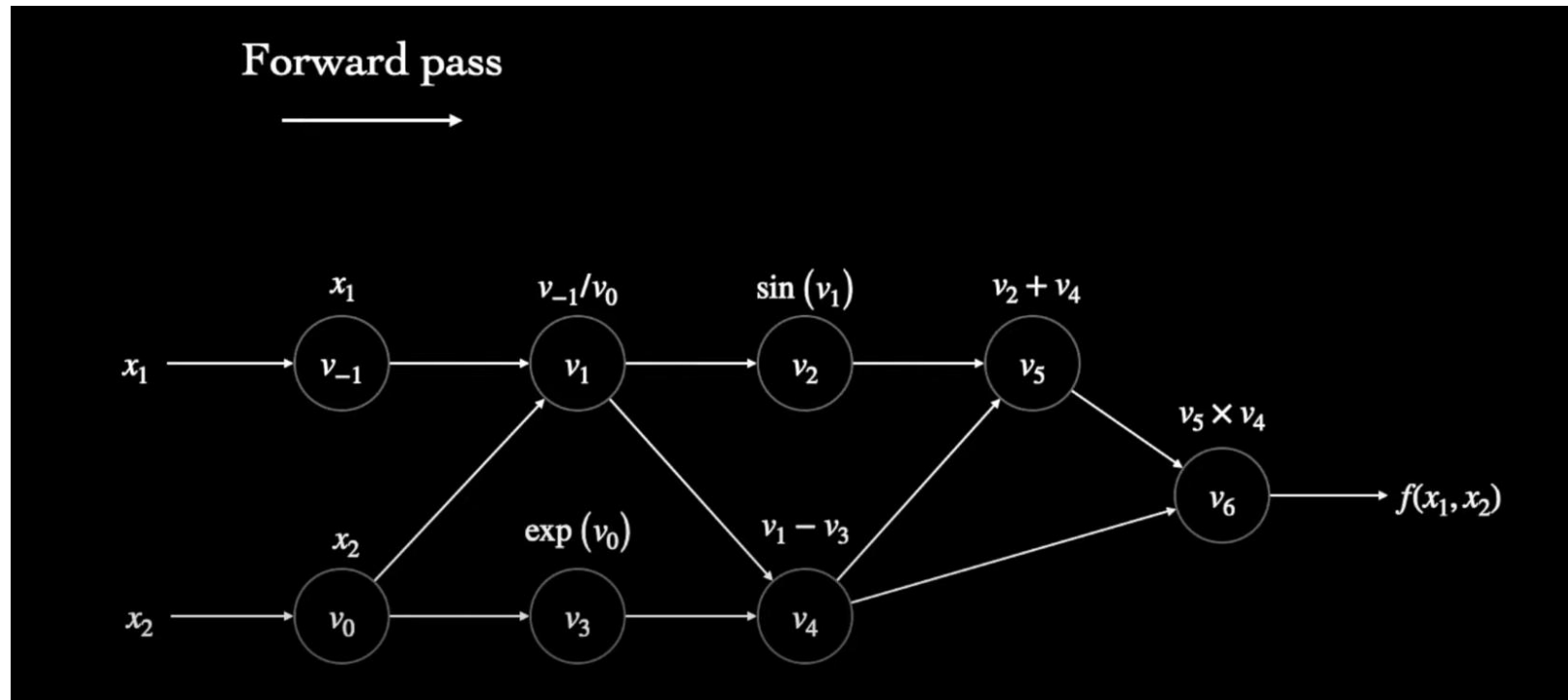
Reverse mode autodiff allows
to handle such scenarios

Differentiation

Automatic Differentiation

Reverse Mode

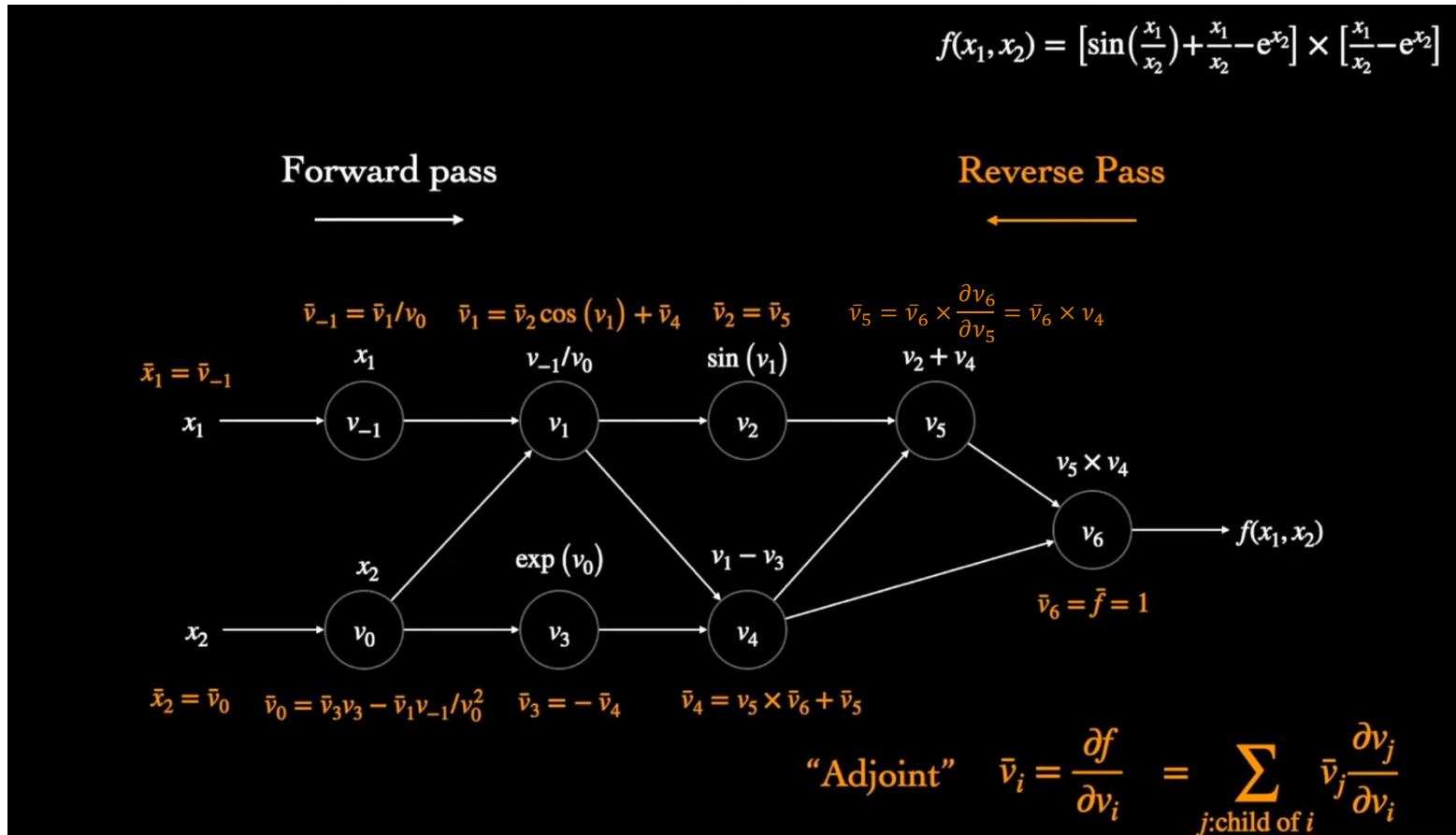
Rather than propagating derivatives forward, they are propagated backward from the output.



During the forward pass, the dependencies of the expression tree are stored in memory. Then, we compute the partial derivatives of the output w.r.t the intermediate variables, known as adjoints.

Differentiation

Automatic Differentiation Reverse Mode



$f: \mathbb{R}^n \rightarrow \mathbb{R}^m$

reverse mode is ideal when $m \ll n$

$$J_f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

- ✓ This way, we obtain partial derivatives w.r.t each input variable.
- ✓ So the gradient is computed with just a single execution of reverse mode *autodiff*. Backpropagation in DNN may be viewed as a special case of reverse mode *autodiff*.

Differentiation

Automatic Differentiation

What about higher order derivatives? Hybrid Forward-Reverse Approach

Forward mode is used to compute the derivatives, so the terms of the **Jacobian**.

Reverse mode is used to differentiate again resulting in the elements of the **Hessian**.

In general, we can compute arbitrarily **higher order derivatives** by **composing multiple executions** of *autodiff* together.

In **PyTorch**, Automatic Differentiation is implemented through the **Autograd** library
TensorFlow provides the **tf.GradientTape** API for Automatic Differentiation

Learning of a solution operator

FNO: Fourier Neural Operator

(Li et al. ICLR 2021)

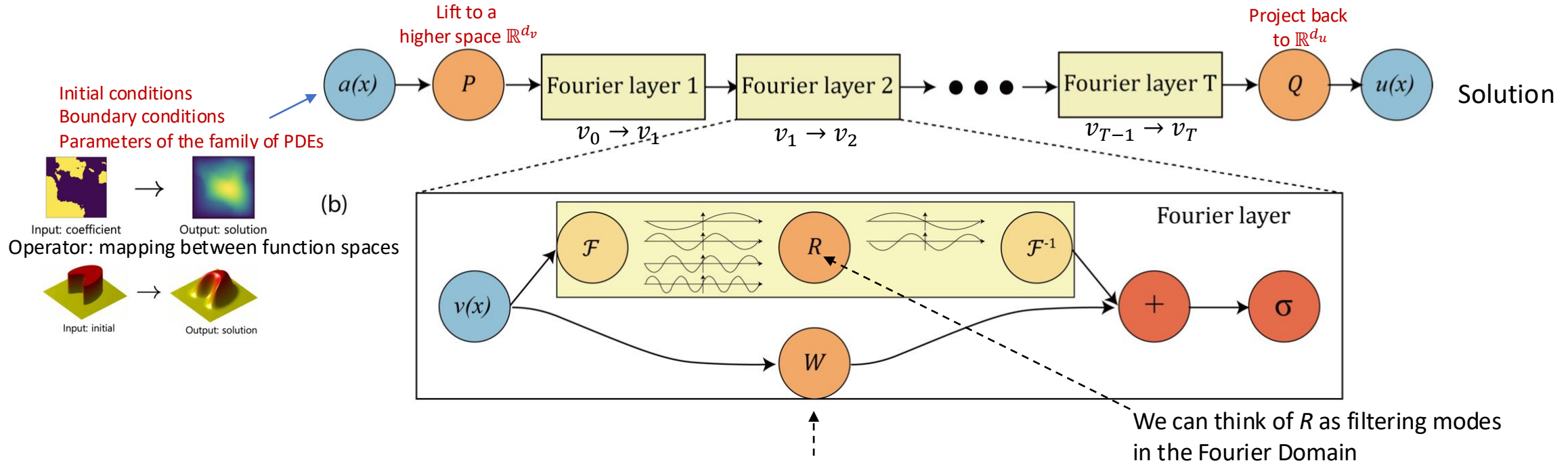
FNO: Fourier Neural Operator

(Li et al. ICLR 2021)

Goal: Unlike PINNS, learn a **solution operator** for a family of PDEs

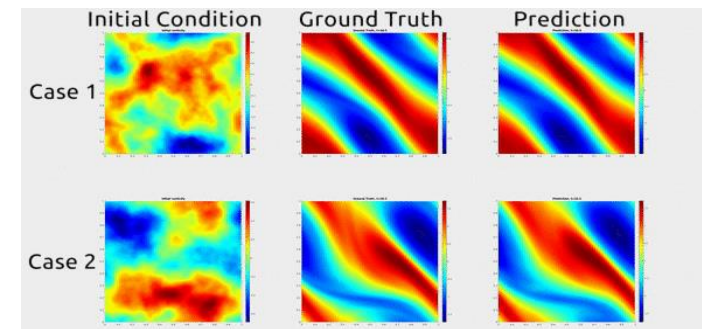
Aims to be resolution invariant by learning **mappings between function spaces**

e.g. images are seen as functions of light defined on a continuous region, instead of as $n \times n$ pixel matrices.



- ✓ The PDE is not supposed to be known
- ✓ Learns a solution operator (not one specific instance)
- ✓ FT is well suited for capturing the global convolution

- ✗ Pairs of (simulation or observation) data are required

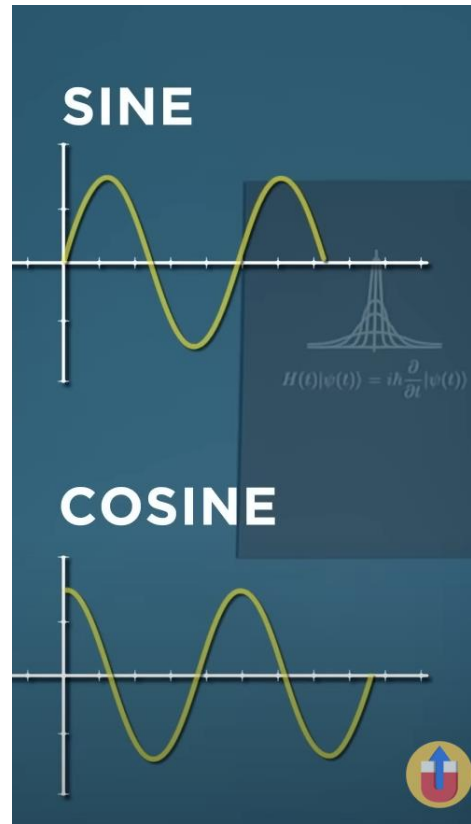
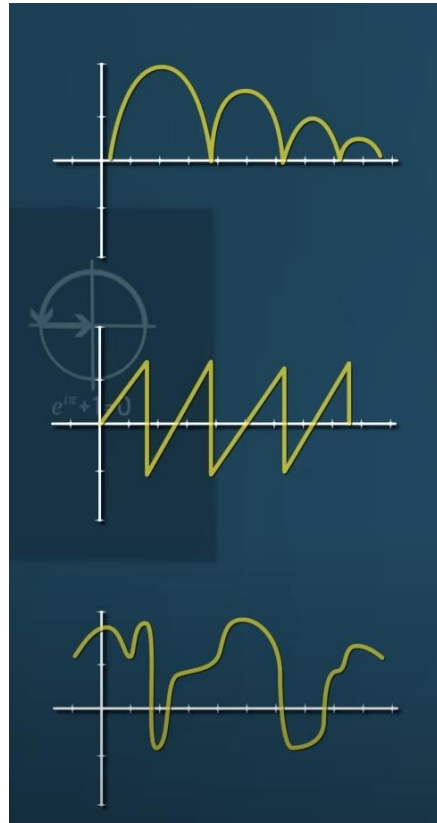


A reminder about

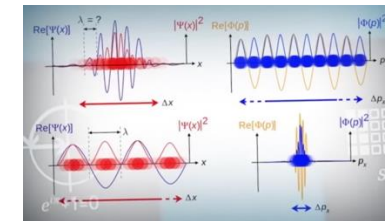
- Fourier series
- Fourier transform
- Fourier and derivatives

Fourier series and Fourier transform

Fourier series principle: Any function can be rewritten by summing up sine and cosine waves



Digital music

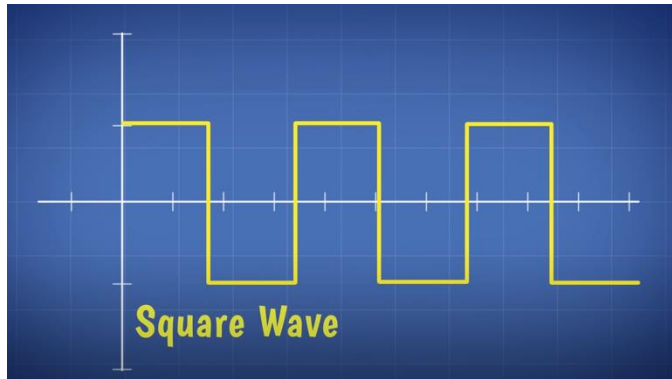


Quantum mechanics

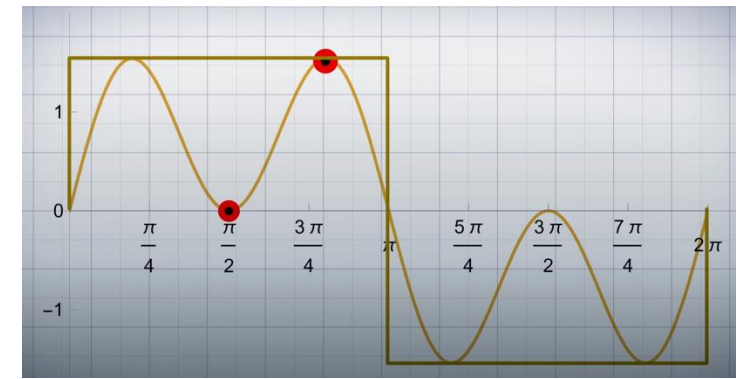
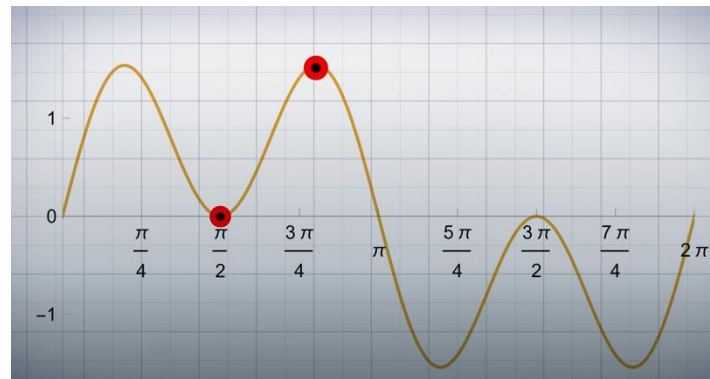
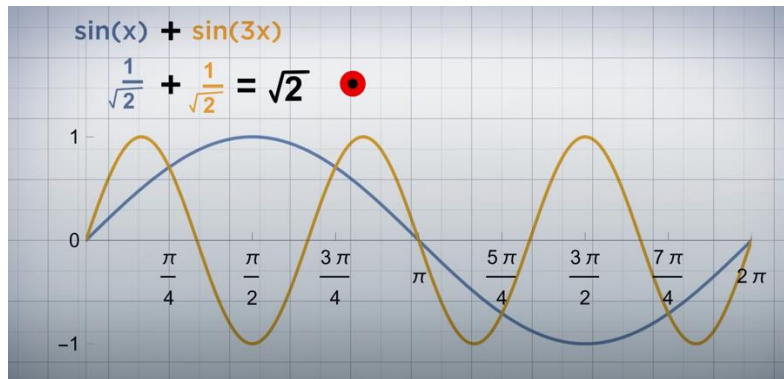
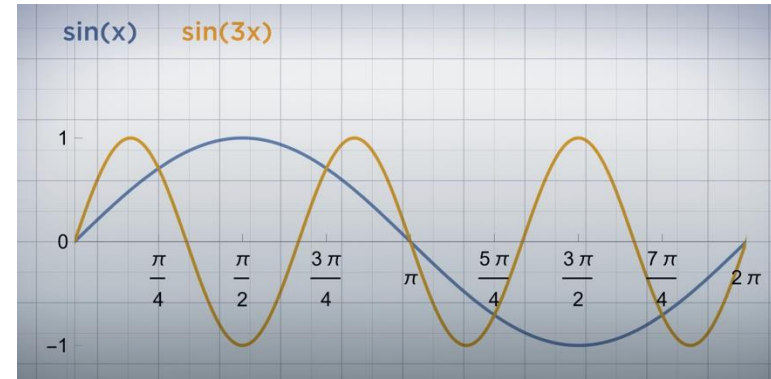


Image recognition

Fourier series: An example

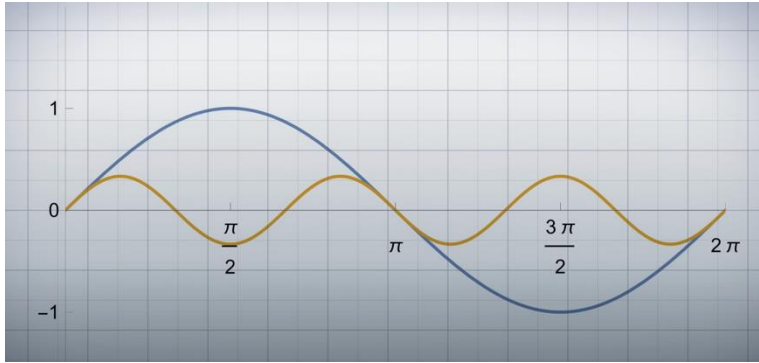


Square wave with sharp corners

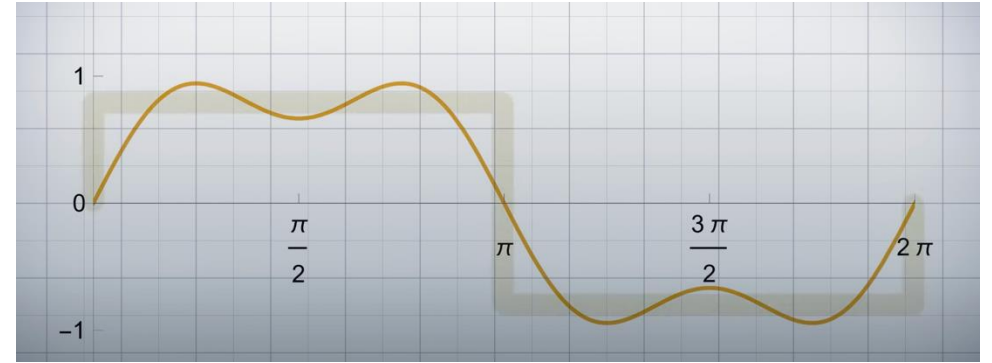


Already look like slightly closer to a square wave

Fourier series: An example



If we give a smaller amplitude to $\sin 3x$



The resulting function is closer to a square wave



By adding more and more terms

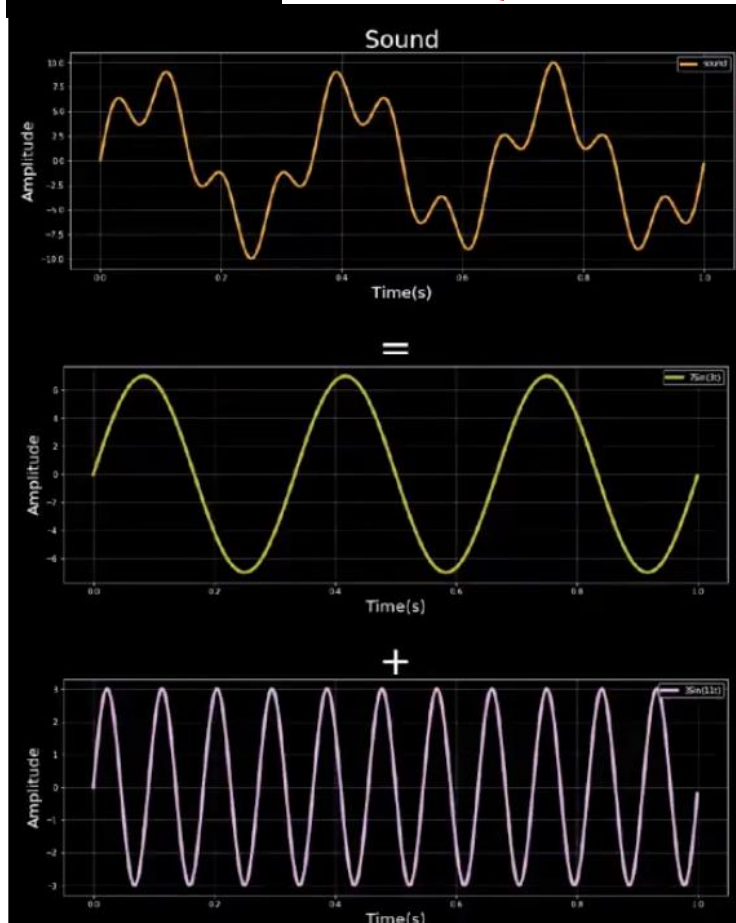


Fourier series are infinite sums
So, we always reach an exact solution

Fourier transform: An example

The Fourier transform converts a function
in time-domain into frequency-domain

Time Domain

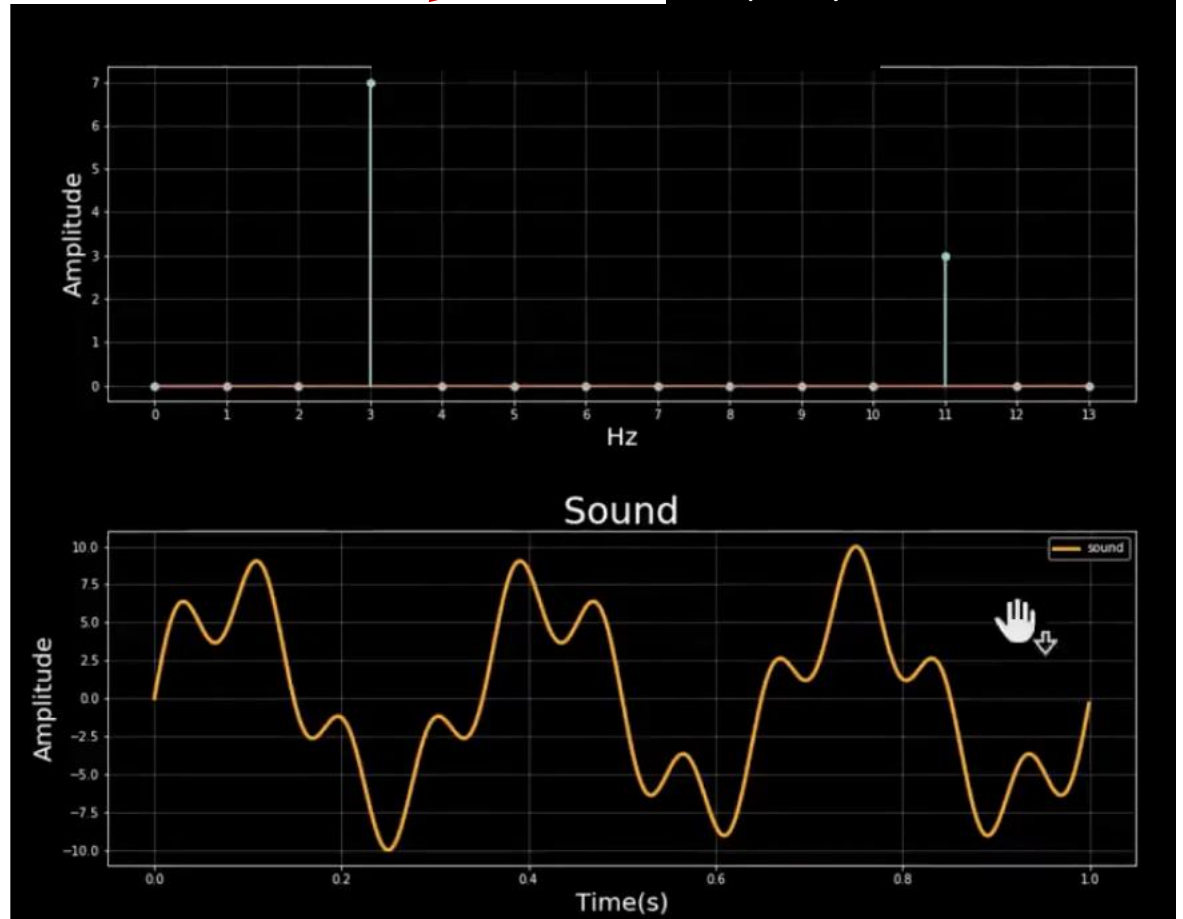


Frequency = 3
Amplitude = 7

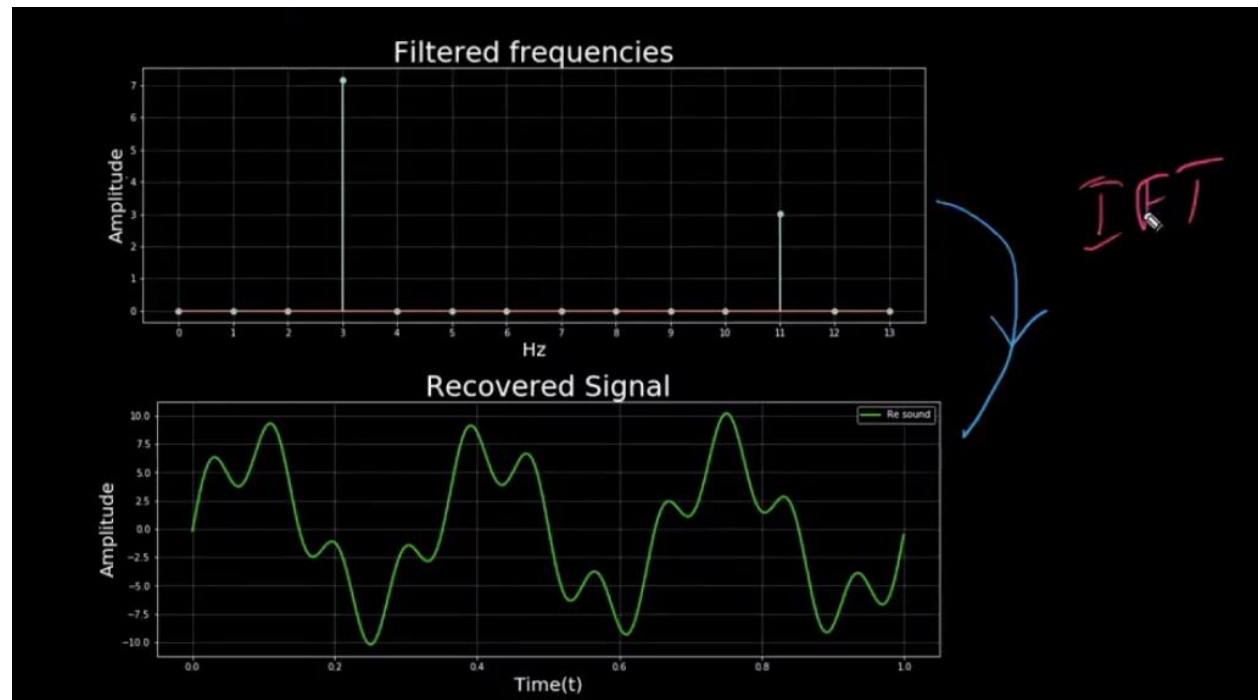
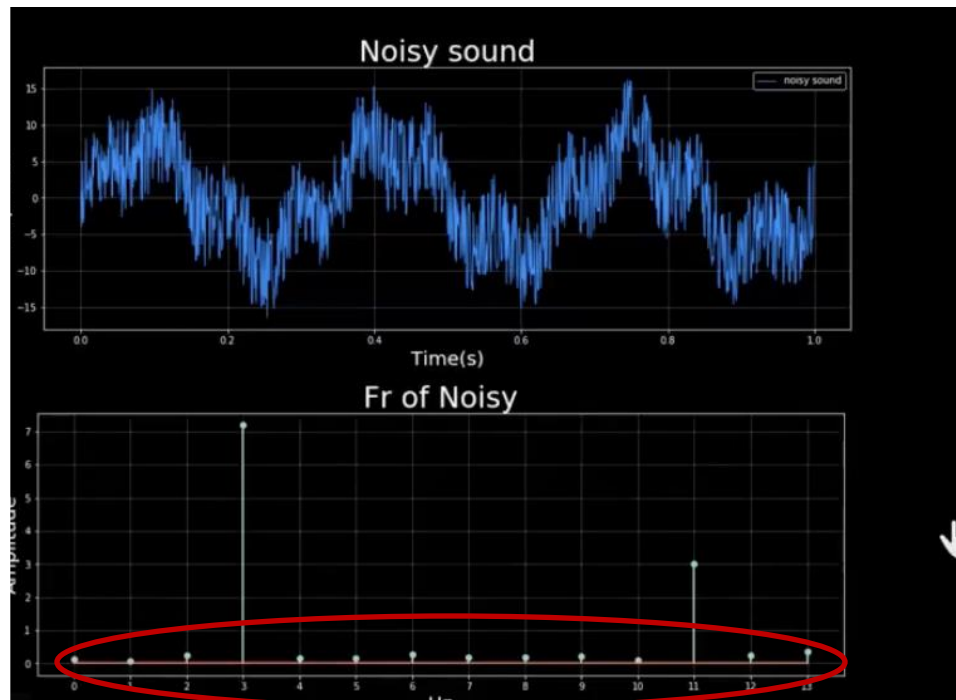
Frequency = 11
Amplitude = 3

2 base frequencies = 3 and 11

Frequency Domain



Noise reduction with Fourier transform and its inverse (IFT)



The Fourier transform and Derivatives

(In practice, we use the Fast Fourier transform (FFT))

$$\hat{f}(\omega) = \mathcal{F}(f(x)) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx$$
$$f(x) = \mathcal{F}^{-1}(\hat{f}(\omega)) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x} d\omega$$

➡ Converts a signal from its original domain (often time or space) to its Fourier transform of a frequency variable ω .

➡ To go back from the frequency domain to the time/space domain

What is particularly useful about the Fourier transform is how it transforms derivatives.

$$\begin{aligned}\mathcal{F}\left(\frac{d}{dx} f(x)\right) &= \int_{-\infty}^{\infty} \underbrace{\frac{df}{dx}}_{dv} \underbrace{e^{-i\omega x}}_u dx \\ &= \underbrace{\left[f(x) e^{-i\omega x} \right]_{-\infty}^{\infty}}_{uv=0} - \int_{-\infty}^{\infty} \underbrace{f(x)}_v \underbrace{(-i\omega e^{-i\omega x})}_{du} dx \\ &= i\omega \underbrace{\int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx}_{\mathcal{F}(f(x))} = \underbrace{i\omega \mathcal{F}(f(x))}_{\mathcal{F}\left(\frac{df}{dx}\right)}\end{aligned}$$

Assuming that $f(x)$
decays to zero at + and - infinity

Integrate by parts

$$\int dv \cdot u = [uv] - \int v du$$

➡ Compute the inverse transform to get the derivative of f
“spectral derivative”

The Fourier transform and...

higher order
derivatives

$$\begin{aligned}
 \mathcal{F}\left(\frac{d}{dx} f(x)\right) &= \int_{-\infty}^{\infty} \underbrace{\frac{df}{dx}}_{dv} \underbrace{e^{-i\omega x}}_u dx \\
 &= \underbrace{\left[f(x) e^{-i\omega x} \right]_{-\infty}^{\infty}}_{uv=0} - \int_{-\infty}^{\infty} \underbrace{f(x)}_v \underbrace{(-i\omega e^{-i\omega x})}_{du} dx \\
 &= i\omega \underbrace{\int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx}_{\mathcal{F}(f(x))} = \underbrace{i\omega}_{\mathcal{F}\left(\frac{df}{dx}\right)} \mathcal{F}(f(x))
 \end{aligned}$$

Suffices to compute $(i\omega)^k$ to get the k th-order derivative

the transformation
of PDEs into ODEs

$$\begin{array}{ccc}
 u_{tt} = c u_{xx} & \xrightarrow{\mathcal{F}} & \hat{u}_{tt} = -\omega^2 \hat{u} \\
 \text{(PDE)} & & \text{(ODE)}
 \end{array}$$

Heat Equation

ODE in the Fourier Domain

2nd order derivative

Fourier transform w.r.t. space, i.e.,

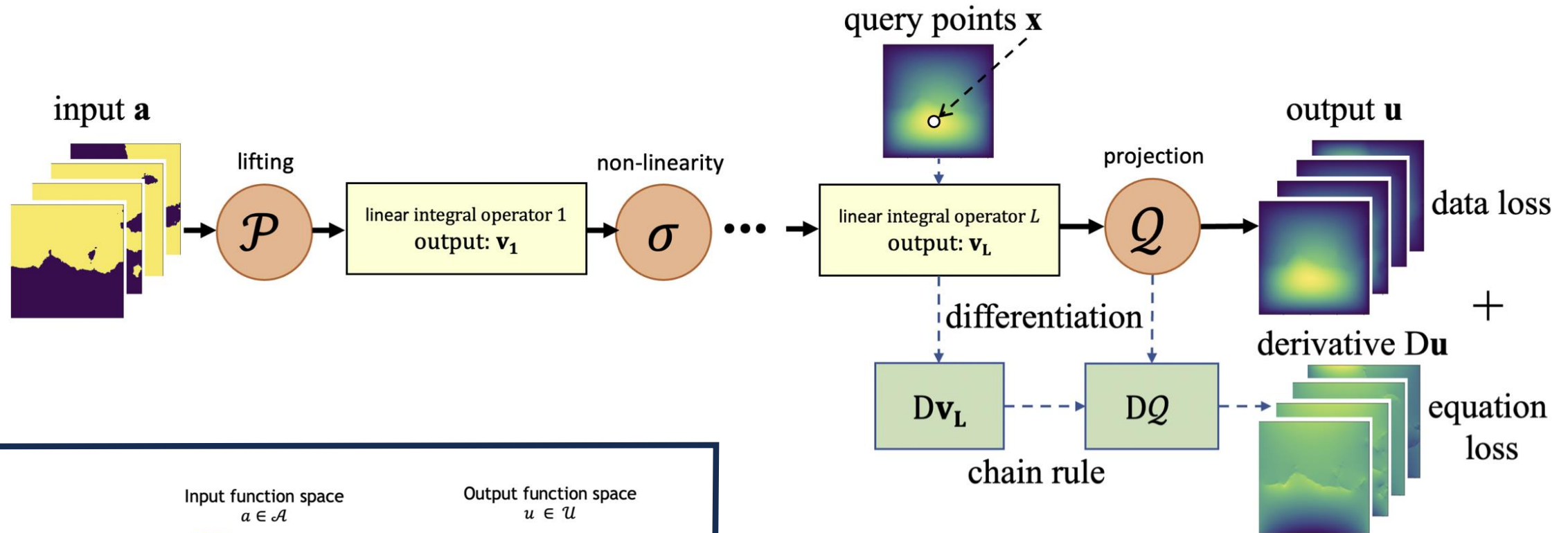
$$u(x,t) \xrightarrow{\mathcal{F}} \hat{u}(\omega,t)$$

PINO: Physics-Informed Neural Operator

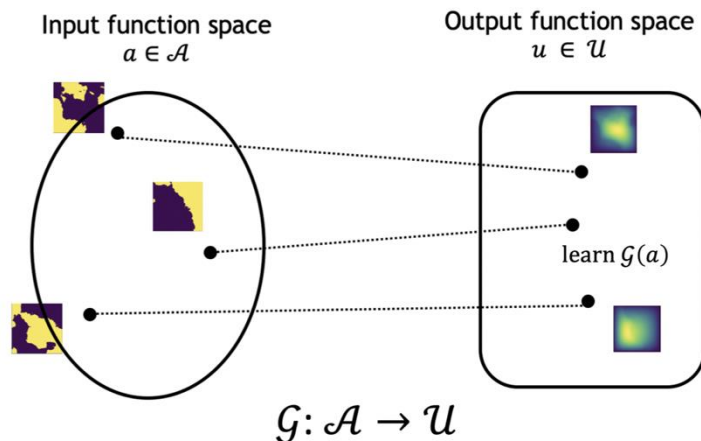
(Li et al. 2023)

Goal: learn the solution operator from **data + physics constraints**

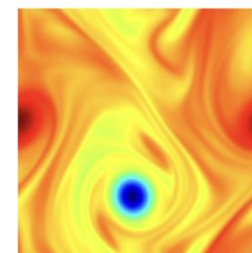
✓ Hybrid (data/physics) model



Like FNO and unlike PINN, PINO learns the solution for a family of PDEs

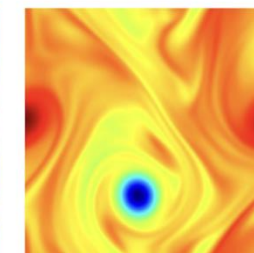


Ground truth



PINO on Kolmogorov flow

Prediction



Outline

1- Introduction

2- Overview of some Physics-informed ML algorithms (PINNs, FNO, PINO, ...)

3- PDE Discovery (SINDy and its variants)

4- Physics-informed ML and Laser-Matter interaction (what we do at Hubert Curien Lab)

Discovering new PDEs

SINDy: Data-driven discovery of PDEs

(Rudy et al. 2017)

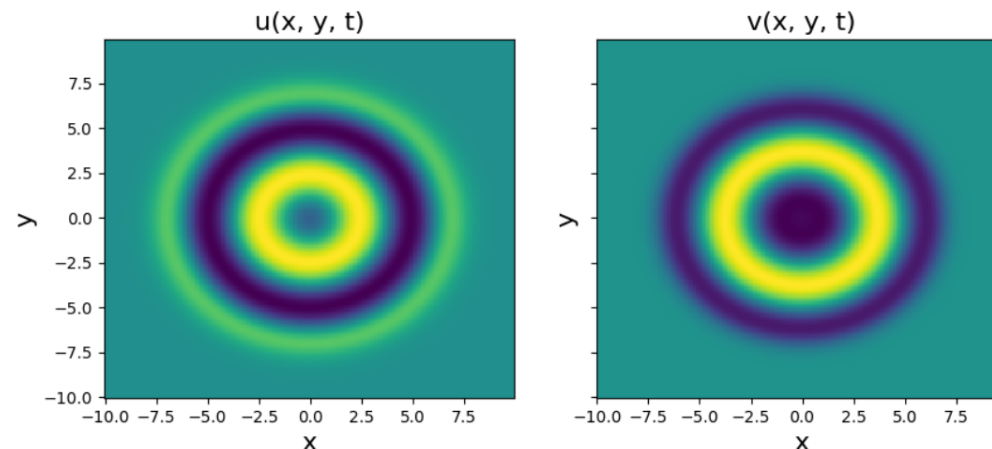
Partial Differential Equations

We consider equations of the general form: $\frac{\partial u}{\partial t} = \mathcal{N}[u]$ where

- $\mathcal{N}[\cdot]$ is a differential operator involving partial space derivatives
- $u(t, \mathbf{x}) \in \mathbb{R}^{d_2}$ is the latent, supposedly unique, hidden solution,
- $t \in [0, T]$ the temporal and $\mathbf{x} \in \Omega \subset \mathbb{R}^d$ the spatial coordinates.

Example: Reaction–diffusion PDEs

$$\begin{cases} u_t &= u - u^3 + v^3 + 0.1u_{xx} + 0.1u_{yy} + u^2v - uv^2 \\ v_t &= v - u^3 - v^3 + 0.1v_{xx} + 0.1v_{yy} - u^2v - uv^2 \end{cases}$$



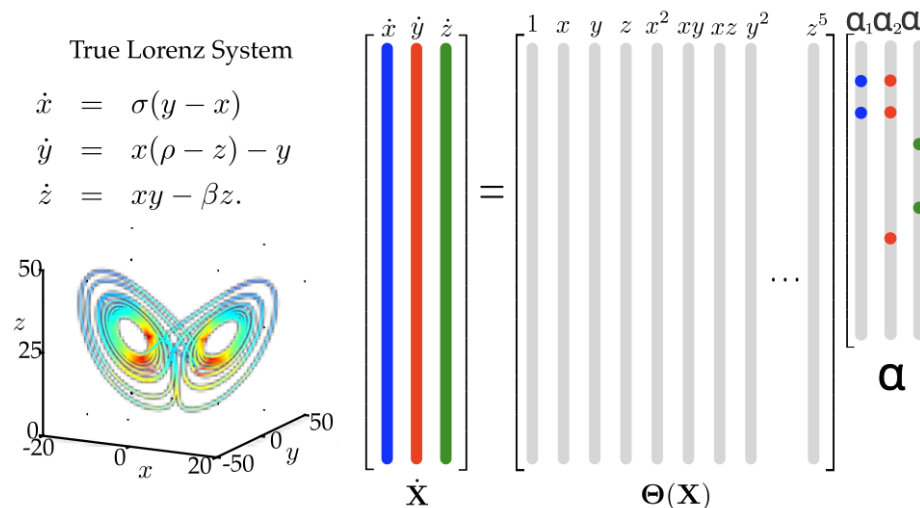
SINDy: Sparse Identification of Nonlinear Dynamics

Realistic assumption

In most equations, only a few important terms govern the underlying dynamics, prompting us to promote sparsity.

SINDy [BPK16a] envisions the problem of learning $\mathcal{N}[u]$ from a **sparse regression perspective** performed from a library Θ of functions (e.g. partial derivatives, trigonometric or polynomial terms, etc.).

$$u_t = \frac{\partial u(t, \mathbf{x})}{\partial t} = \Theta_u(t, \mathbf{x}) \alpha$$



SINDy: Explicit method for approximating $\frac{\partial u(t, \mathbf{x})}{\partial t}$

$$u_t = \Theta_u(t, \mathbf{x})\alpha \quad (1)$$

In explicit techniques, like Euler or Runge Kunta numerical methods, $u_t = \frac{\partial u(t, \mathbf{x})}{\partial t}$ is estimated using finite difference schemes.

Euler-SINDy

$$\tilde{u}_t = \underbrace{\frac{u(t_{j+1}, \mathbf{x}_m) - u(t_j, \mathbf{x}_m)}{h_j}}_{\text{Euler estimate}} \quad (2)$$

Equating (1) and (2), we get:

$$u(t_{j+1}, \mathbf{x}_m) = u(t_j, \mathbf{x}_m) + h_j \cdot \Theta_u(t, \mathbf{x})\alpha \quad (3)$$

Eq. (3) can then be used to define the loss function of Euler-SINDy.

Euler-SINDy: Optimization problem

Euler-SINDy aims to solve the following problem from $N = J \times M$ pairs:

$$\min_{\alpha} \sum_{j=0}^{J-1} \sum_{m=1}^M \left\| u(t_{j+1}, \mathbf{x}_m) - \underbrace{\left(u(t_j, \mathbf{x}_m) + h_j \cdot \Theta_u(t_j, \mathbf{x}_m) \alpha \right)}_{\text{Euler estimate}} \right\|_2^2 + \lambda \|\alpha\|_{\mathcal{F}}^2,$$

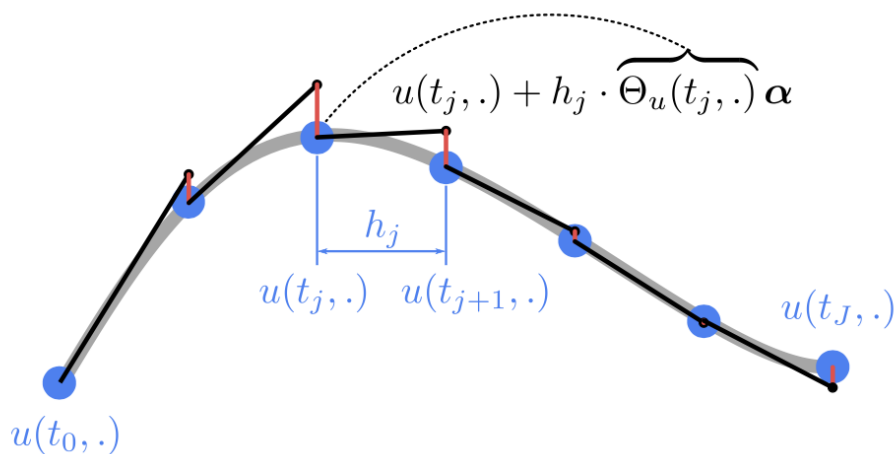


Figure: Dense data: truncation error ↘

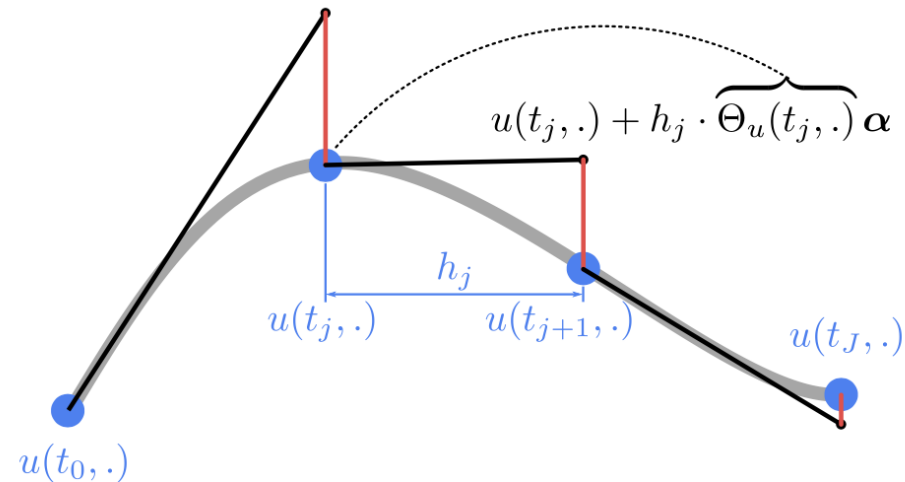


Figure: Scarce data: truncation error ↗

Local truncation error

Local truncation error of Euler' numerical method is on the order of $\mathcal{O}(h^2)$

Euler-SINDy: Optimization problem

Euler-SINDy aims to solve the following problem from $N = J \times M$ pairs:

$$\min_{\alpha} \sum_{j=0}^{J-1} \sum_{m=1}^M \left\| u(t_{j+1}, \mathbf{x}_m) - \underbrace{\left(u(t_j, \mathbf{x}_m) + h_j \cdot \Theta_u(t_j, \mathbf{x}_m) \alpha \right)}_{\text{Euler estimate}} \right\|_2^2 + \lambda \|\alpha\|_{\mathcal{F}}^2,$$

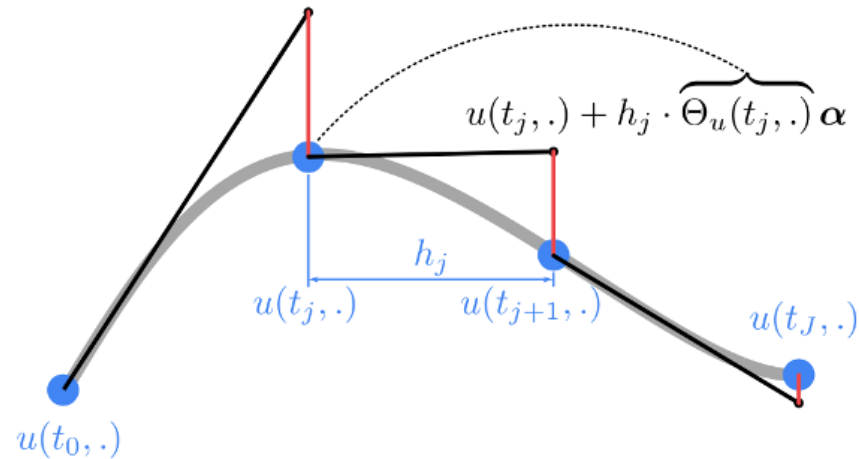


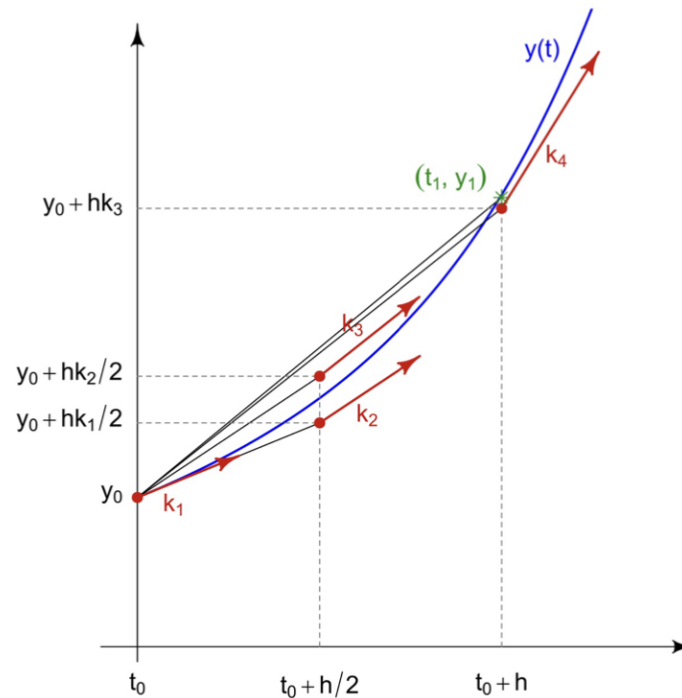
Figure: Scarce data: truncation error ↗

The coefficients of the PDE (ground truth) are not minimizers of this optimization problem!

The **Runge-Kutta approximation** improves Euler method by starting with an initial estimate of the solution and using the latter to calculate a second, more accurate approximation, and so on. In RK4 (with error $\sim \mathcal{O}(h^5)$):

$$u(t + h_t) = u(t) + \frac{h_t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

where $k_1 = f(t, u(t))$, $k_2 = f(t + \frac{h_t}{2}, u(t) + h_t \frac{k_1}{2})$,
 $k_3 = f(t + \frac{h_t}{2}, u(t) + h_t \frac{k_2}{2})$ and $k_4 = f(t + h_t, u(t) + h_t k_3)$.



SINDy: Variants and Extensions

Limitation of Explicit Methods

Explicit methods rely on accurate time derivative approximations, imposing constraints on the sampling time step sizes h of the data especially when solving stiff equations.

- **SINDYc** [BPK16b] to leverage external inputs and feedback control,
- **Reactive SINDy** [HFN18] to deal with vector-valued ansatz functions,
- **Ensemble-SINDy** [FKBB22] enabling uncertainty quantification,
- **SINDy-PI** [KKB20] which uses rational functions to discover PDE,
- Extension of SINDy to **stochastic dynamical systems** [BNC18]
- **WSINDy** [MB21] which eliminates pointwise derivative approximations with a weak formulation providing better robustness to noise.

Key Question

How to decorrelate the numerical integration time step from the inter-observation time step h ? → **Unrolling** K times the approximation.

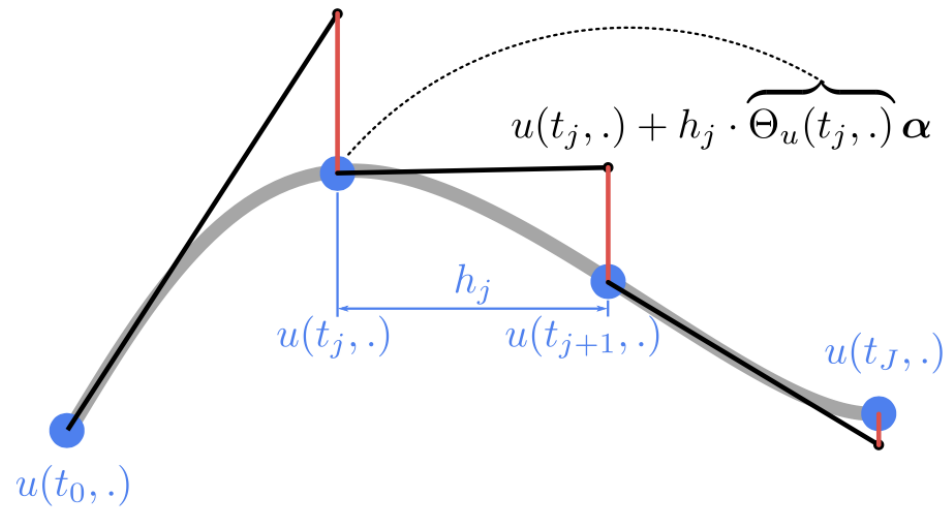


Figure: Scarce data: truncation error ↗

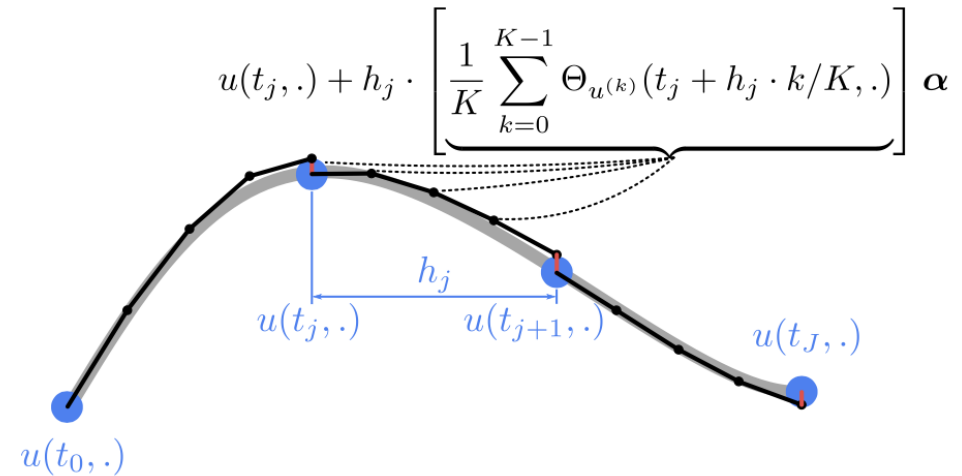


Figure: K-Unrolled Euler-SINDy ($K=4$)

$$u^{(K)}(t_j + h_j, .) = u(t_j, .) + h_j \cdot \underbrace{\left[\frac{1}{K} \sum_{k=0}^{K-1} \Theta_{u^{(k)}}(t_j + k \cdot h_j/K, .) \right]}_{\text{K-Unrolled Euler estimate}} \cdot \alpha \quad (4)$$

K unrolled Euler benefits implicitly from an error of the order $\mathcal{O}\left(\frac{h^2}{K}\right)$

Theorem (Local truncation error of K unrolled Euler)

The local truncation error suffered by the unrolled Euler estimate of Eq. (4) (assuming that $\forall j, h_j = h$) is on the order $\mathcal{O}\left(\frac{h^2}{K}\right)$ such that:

$$\epsilon = \left(\frac{h^2}{2K}\right) \cdot \left| \frac{1}{K} \sum_{i=0}^{K-1} u''\left(t + \frac{hi}{K}\right) \right| \leq \left(\frac{h^2}{2K}\right) \cdot M, \quad (5)$$

with the constant $M = \max_{t' \in [t, t+h]} |u''(t')|$.

This theorem states that as K tends to infinity, ϵ converges towards 0. This means that, provided that the dictionary Θ contains the correct terms, there exists an α in Eq. (4) that corresponds to the governing equation and which allows a correct prediction of the next observation.

Theorem (Local truncation error of K -unrolled Runge Kutta)

Assume that $|\frac{\partial f}{\partial u}| \leq L$. Then there exists $C \in \mathbb{R}^+$ such that the error ϵ of a K -unrolled one-step of an s -stage Runge-Kutta method of order p , with a time-step h satisfies:

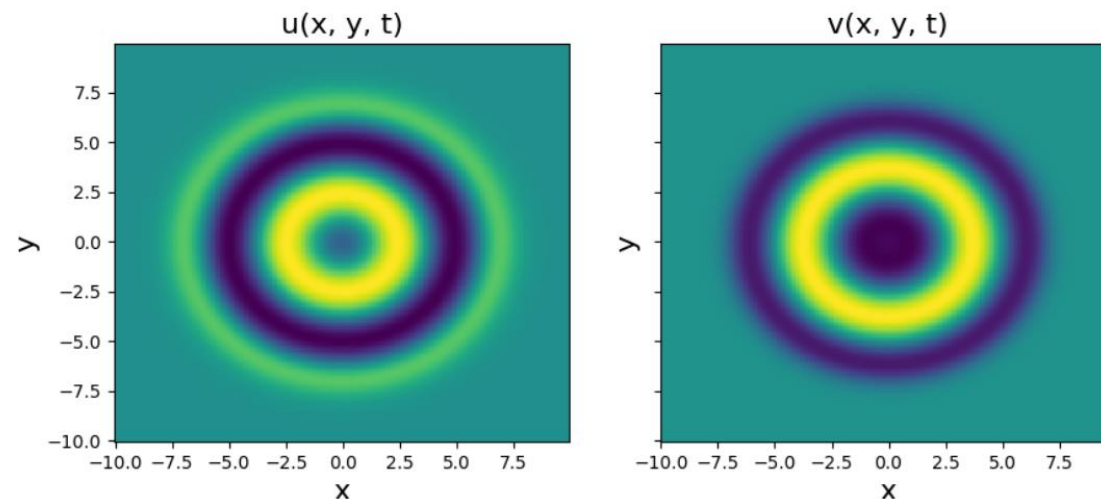
$$\epsilon \leq \left(\frac{h}{K}\right)^p \frac{C}{L} \left(e^{Lh} - 1\right) \quad (6)$$

Note that for Euler's method, *i.e.* $p = 1$, this theorem holds since the right hand part of the inequality of Eq.(5) is upper bounded by that of Eq.(6).

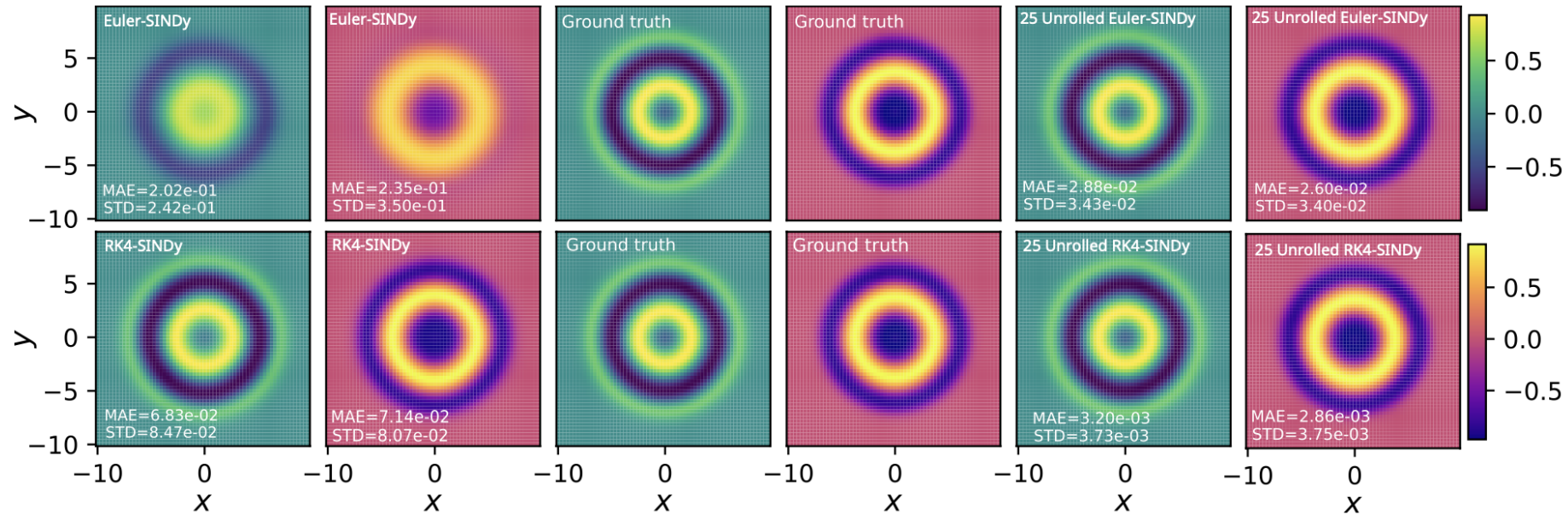
2D Reaction-Diffusion PDEs

Reaction–diffusion PDEs model physical phenomena such as the change in space and time of the concentration chemical substances. The dynamics is as follows:

$$\begin{cases} u_t &= u - u^3 + v^3 + 0.1u_{xx} + 0.1u_{yy} + u^2v - uv^2 \\ v_t &= v - u^3 - v^3 + 0.1v_{xx} + 0.1v_{yy} - u^2v - uv^2 \end{cases}$$



2D Reaction-Diffusion PDEs



$$\text{Ground Truth} \begin{cases} u_t = u - u^3 + v^3 + 0.1u_{xx} + 0.1u_{yy} + u^2v - uv^2 \\ v_t = v - u^3 - v^3 + 0.1v_{xx} + 0.1v_{yy} - u^2v - uv^2 \end{cases}$$

$$\text{Euler-SINDy} \begin{cases} u_t = 1.11u - 1.47u^3 + 0.83v^3 + 0.21u_{xx} + 0.21u_{yy} + 0.88u^2v - 1.41uv^2 (+4) \\ v_t = 0.33v - 0.64u^3 + 0.07v_{xx} + 0.07v_{yy} - 0.64u^2v (+1) \end{cases}$$

$$\text{Unr. Euler-SINDy} \begin{cases} u_t = 1.00u - 1.01u^3 + 1.01v^3 + 0.11u_{xx} + 0.11u_{yy} + 1.01u^2v - 1.01uv^2 \\ v_t = 1.01v - 1.01u^3 - 1.02v^3 + 0.11v_{xx} + 0.11v_{yy} - 1.02u^2v - 1.00uv^2 \end{cases}$$

Outline

1- Introduction

2- Overview of some Physics-informed ML algorithms (PINNs, FNO, PINO, ...)

3- PDE Discovery (SINDy and its variants)

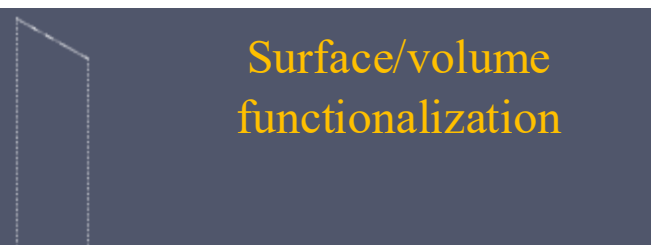
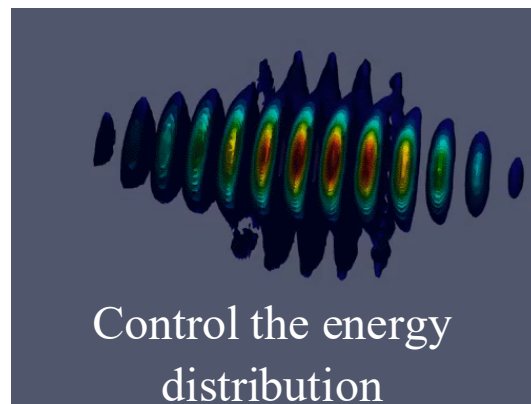
4- Physics-informed ML and Laser-Matter interaction (what we do at Hubert Curien Lab)

Laser-matter interactions

Modeling the space-time dynamics of ultrafast laser beams

Ultrafast laser beam

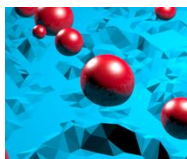
Material modifications



$\div 10\,000\,000$



$\div 10\,000\,000$

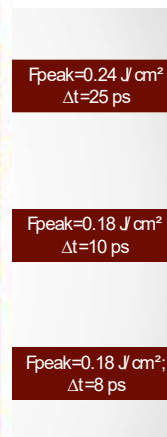
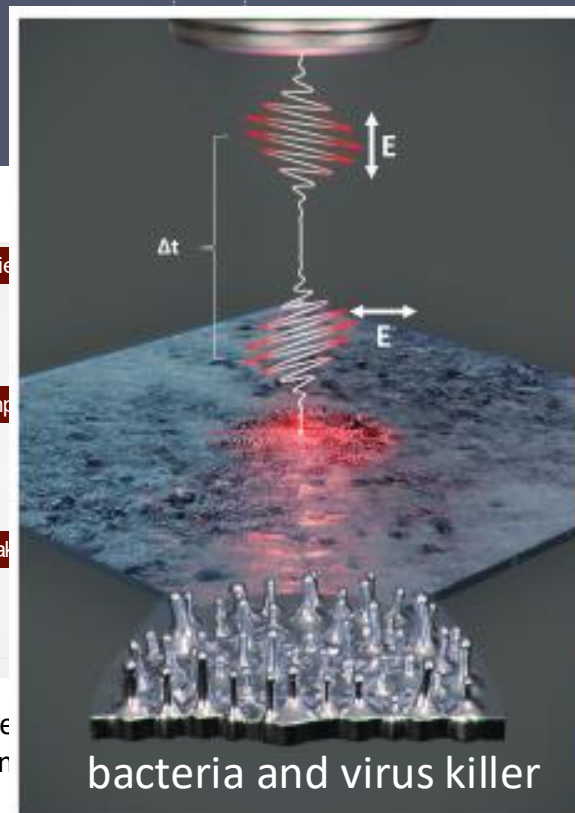


Nanocavities

Nanobumps

Nanopillars

These ground



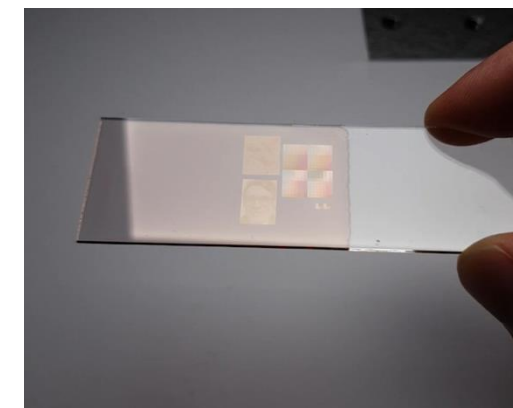
ly applications.



Transmission

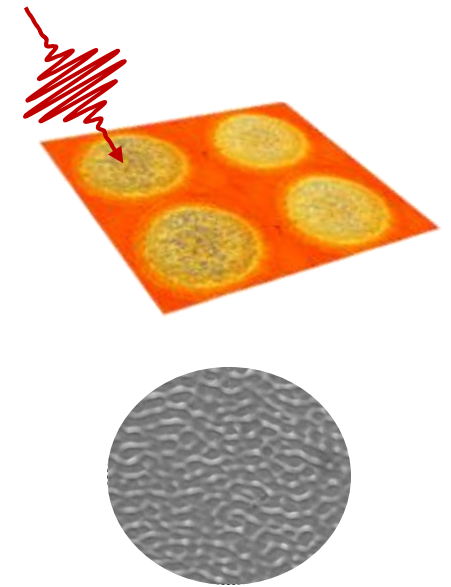
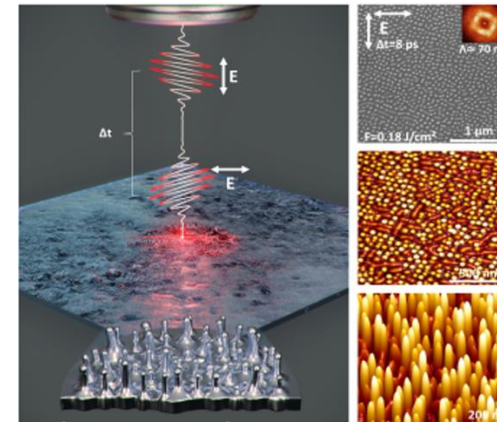
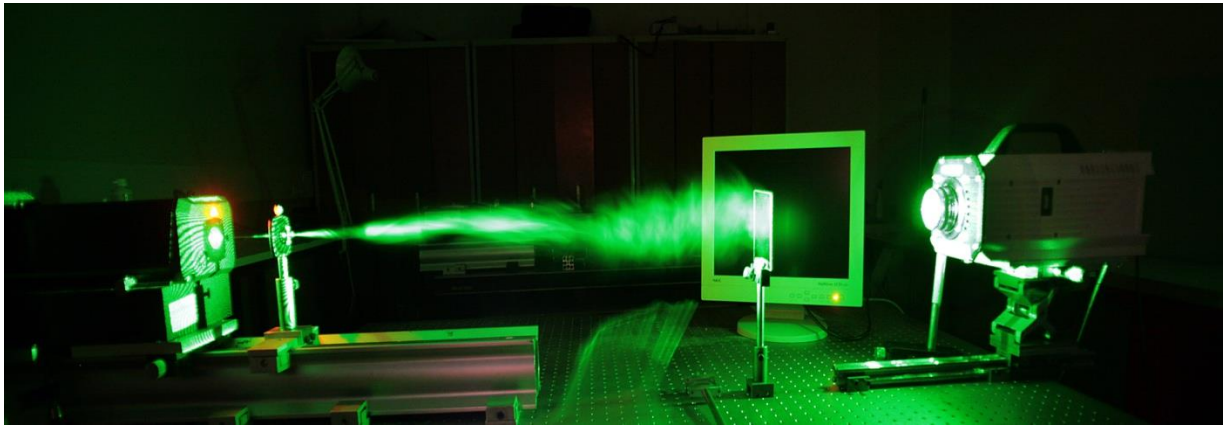


Reflexion



Laser-Matter Interaction

Self-organization of matter



What is self-organization?

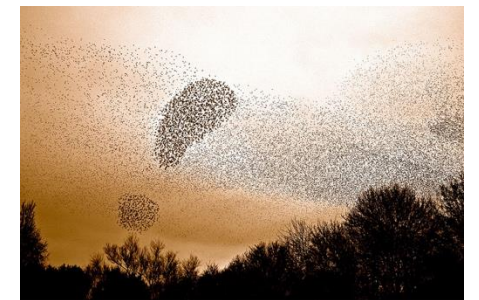
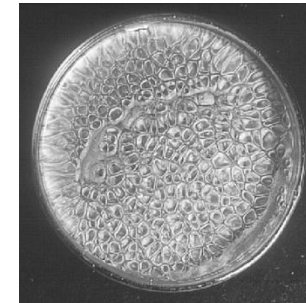
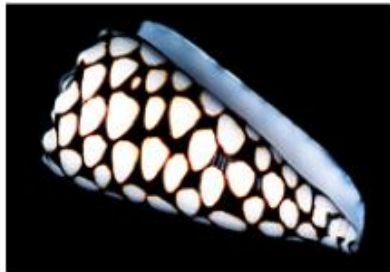
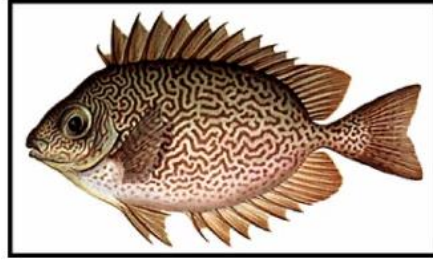
“...corresponds to the emergence of patterns obtained from numerous local interactions.”

Applications:

- Health
- Energy
- Security
- etc.

Examples of self-organization in nature

Behind a pattern = a function



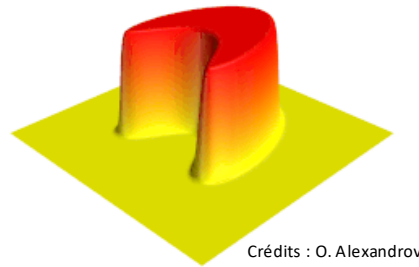
Physical knowledge and Machine Learning

Some physics are well known...



$$\begin{aligned}\dot{u} + (u \cdot \nabla)u &= -\nabla p + \nu \Delta u + f \\ \nabla \cdot u &= 0\end{aligned}$$

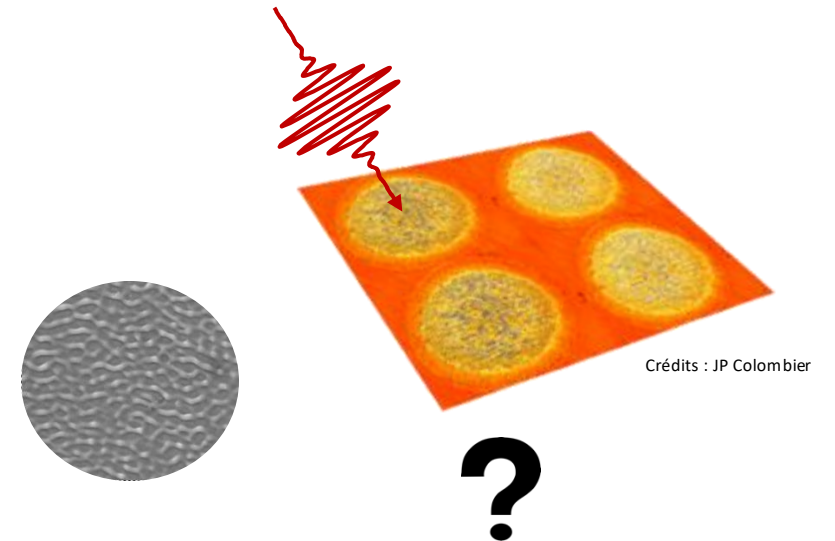
Navier-Stokes Equation
Fluid mechanics



$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right)$$

Heat Equation
Heat conduction

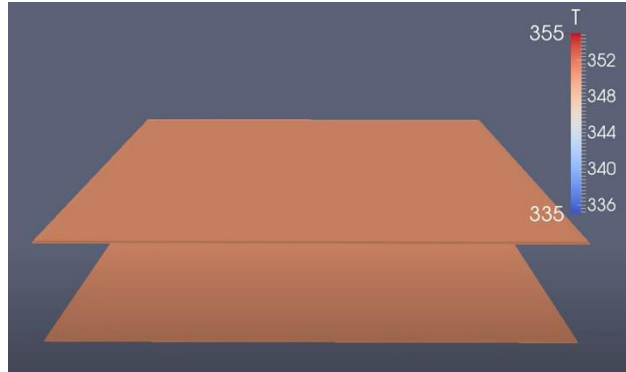
... others are more difficult to model



Laser-matter Interaction

Modeling self-organization?

Much harder than other dynamical systems



Rayleigh–Bénard convection

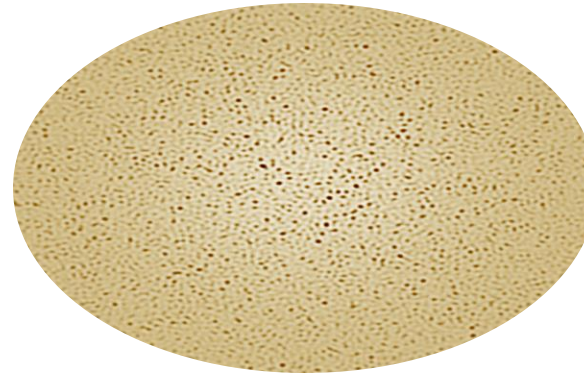
(the density of particles near the lower surface decreases)



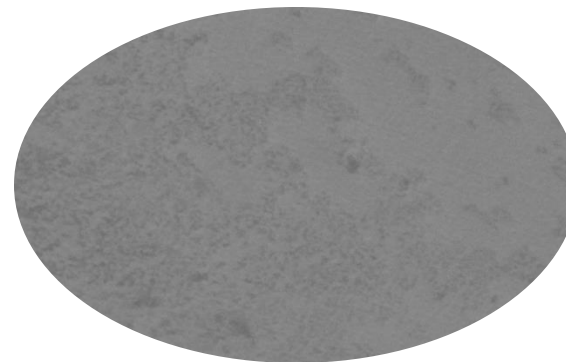
Convection in soup!

Swift-Hohenberg Equation

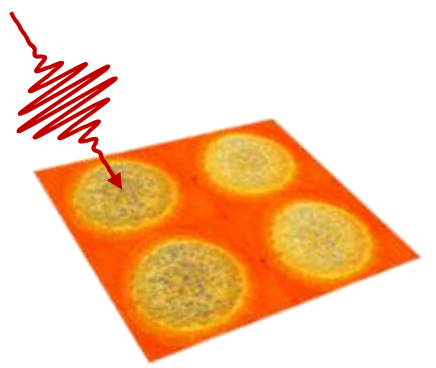
$$\frac{\partial u}{\partial t} = (\epsilon - 1)u - 2\frac{\partial^2 u}{\partial^2 x} - \frac{\partial^4 u}{\partial^4 x} + u^3 - \gamma u^2;$$



Simulated data

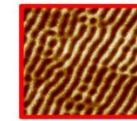


Experimental data

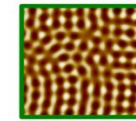


Stripes

real

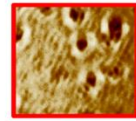


gen

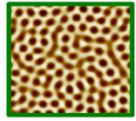
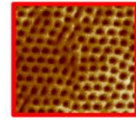
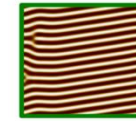
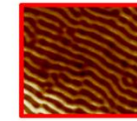
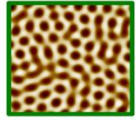
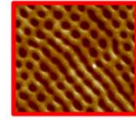
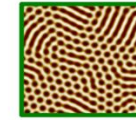
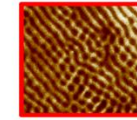
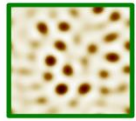


Hexagons

real



gen



Scanning Electron Microscope (SEM) images SH PDE

Main challenges:

- Theoretical issues:
 - 4th order PDE
 - Non linear PDE
- Algorithmic issues:
 - Partially explains the dynamics
 - Lack of observation data



Data is scarce...in laser-matter interaction



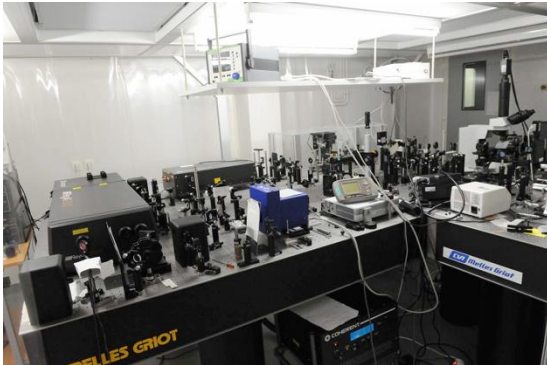
(Physicists) : “How many data do you need to learn well?”



(Machine learner) : “More than you can possibly give us...”



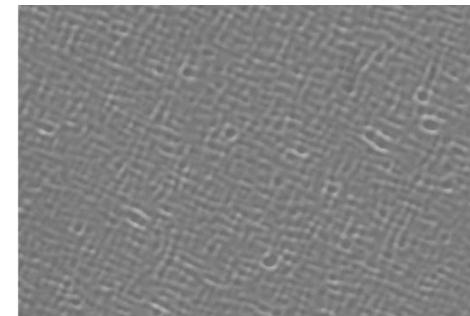
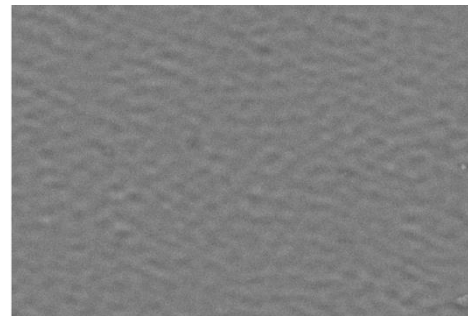
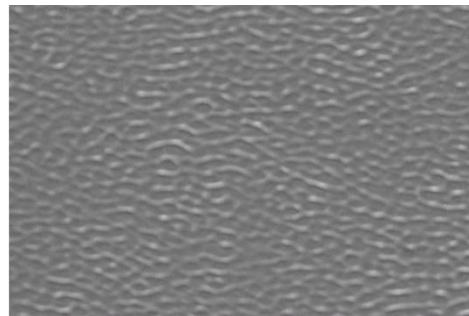
The access to huge amount of data
is difficult



Femtosecond laser platform

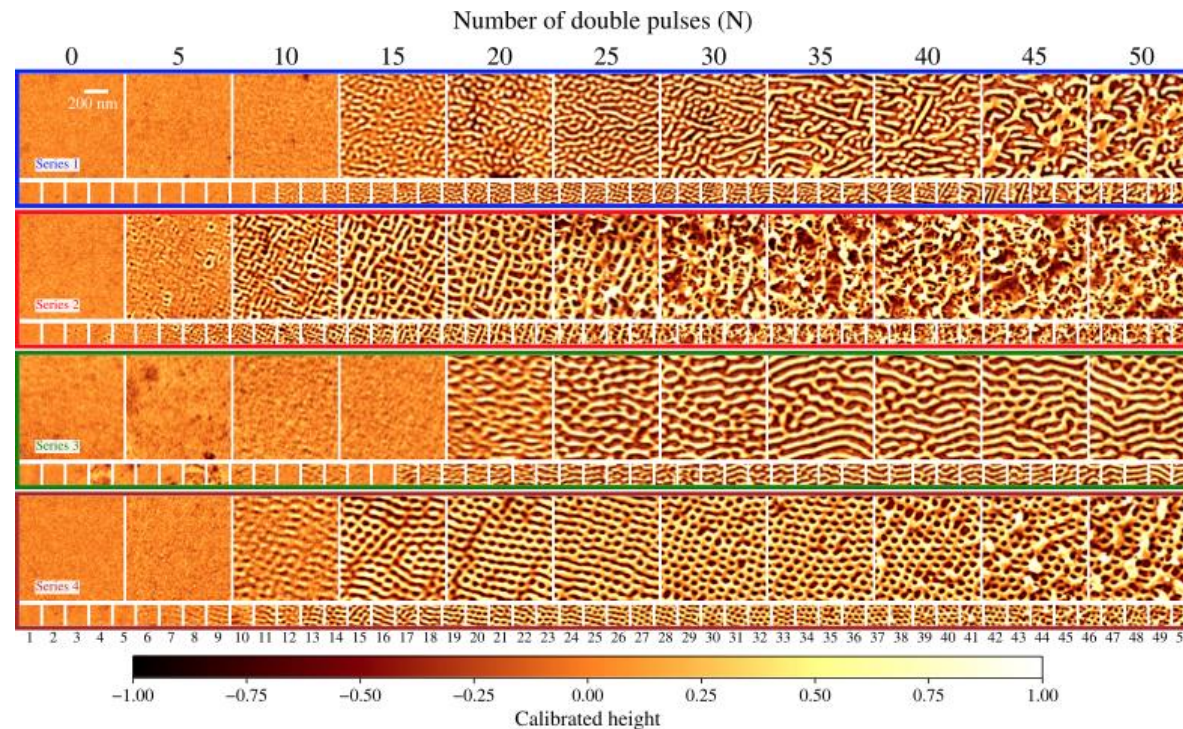
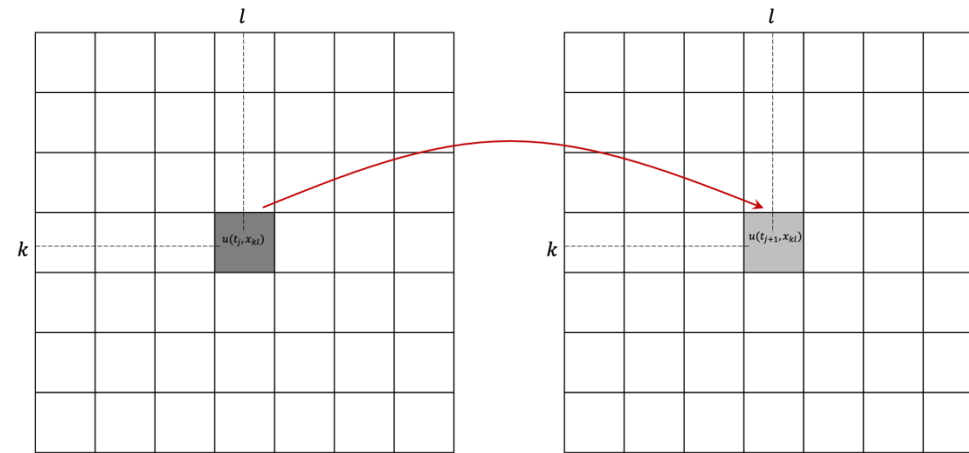


Scanning electron
microscope



Images are unregistered...in laser-matter interaction

Light propagates too fast and the pulse is too short (100 femtosecond) to observe the whole dynamics (e.g. unlike in climate).



(Data+knowledge)-driven PINNs in Laser-Matter Interaction

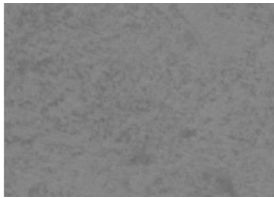
Exploration of the *pattern/function space*

Partial physical knowledge

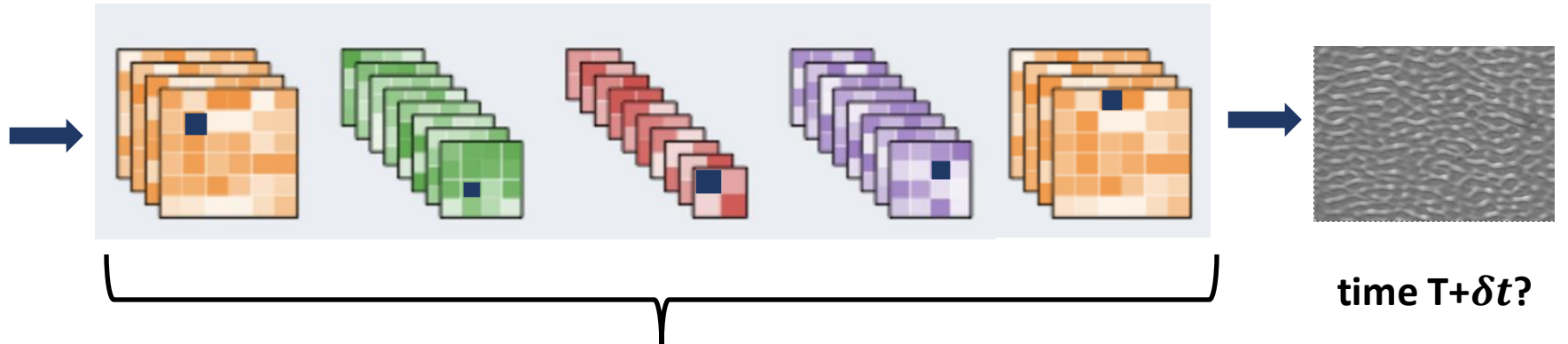
$$\frac{\partial u}{\partial t} + \mathcal{N}[u; \phi] = 0$$

+

a few labeled data



time T



time T+ δt ?

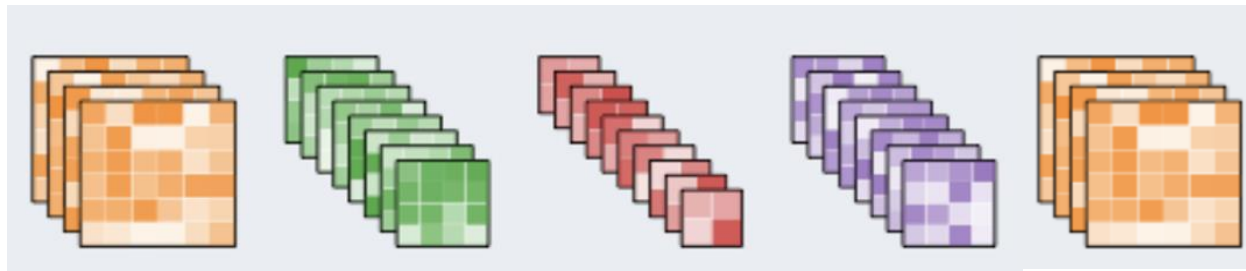
Discovery of new patterns/functions

Even better.... Solving the inverse problem

Star tortoise



Energy E ?
Nb of laser pulses N ?
Delay δt ?



Functions?

- Allows to straighten up when on the back
- Optimal for heat storage

What are the laser parameters needed
for a certain function/pattern?

If you want to play with PDE learning...

Software name	Usage	Language	Backend
DeepXDE	Solver	Python	TensorFlow
SimNet	Solver	Python	TensorFlow
PyDEns	Solver	Python	TensorFlow
NeuroDiffEq	Solver	Python	PyTorch
NeuralPDE	Solver	Julia	Julia
SciANN	Wrapper	Python	TensorFlow
ADCME	Wrapper	Julia	TensorFlow
GPyTorch	Wrapper	Python	PyTorch
Neural Tangents	Wrapper	Python	JAX



Inria team MALICE : <https://labhc-malice.github.io>

